

# Multi-Objective Loss Balancing for Physics-Informed Deep Learning<sup>☆</sup>

Rafael Bischof<sup>a</sup>, Michael A. Kraus<sup>b</sup>,\*

<sup>a</sup> Computational Design Lab, ETH Zürich, Zürich, Switzerland

<sup>b</sup> Chair of Structural Analysis, Institute of Structural Mechanics and Design (ISMD), TU Darmstadt, Darmstadt, Germany

## ARTICLE INFO

### Keywords:

Partial differential equations  
Scientific machine learning  
Helmholtz equation  
Burgers equation  
PINNacle  
Loss balancing  
Multi-objective optimisation

## ABSTRACT

Physics-Informed Neural Networks (PINN) are deep learning algorithms that leverage physical laws by including partial differential equations together with a respective set of boundary and initial conditions as penalty terms in their loss function. In this work, we observe the significant role of correctly weighting the combination of multiple competitive loss functions for training PINNs effectively. To this end, we implement and evaluate different methods aiming at balancing the contributions of multiple terms of the PINN's loss function and their gradients. After reviewing three existing loss scaling approaches (Learning Rate Annealing, GradNorm and SoftAdapt), we propose a novel self-adaptive loss balancing scheme for PINNs named *ReLoBRaLo* (Relative Loss Balancing with Random Lookback). We extensively evaluate the performance of the aforementioned balancing schemes by solving both forward as well as inverse problems on three benchmark PDEs for PINNs: Burgers' equation, Kirchhoff's plate bending equation, Helmholtz's equation and over 20 PDEs from the "PINNacle" collection. The results show that *ReLoBRaLo* is able to consistently outperform the baseline of existing scaling methods in terms of accuracy while also inducing significantly less computational overhead for a variety of PDE classes.

## 1. Introduction

The emergence of Physics-Informed Neural Networks (PINNs) [1] has sparked a lot of interest in domains that see themselves regularly confronted with problems in the low data regime. By leveraging well-known physical laws and incorporating them as implicit prior into the deep-learning pipeline, PINNs were shown to require little to no data in order to approximate partial differential equations (PDE) of varying complexity [1,2].

We consider the case where PINNs are used to find the unknown underlying function that is proper for solving a parameterised PDE. PDEs generally consist of a governing equation and a set of boundary and initial conditions. When trained jointly, i.e. as multi-objective optimisation (MOO), these equations form a set of objective functions that guide the model to approximate a function that satisfies the PDE. While several established and well-studied numerical methods already exist for addressing this problem, such as the Finite Element Method (FEM), Finite Difference Method or Wavelets resp. Laplace transform methods [3–6], PINNs offer significant advantages, such as being end-to-end differentiable, mesh-free and avoiding the curse of dimensionality [7,8]. They could, therefore, prove useful in several engineering applications, such as inversion and surrogate modelling in solid mechanics [9],

<sup>☆</sup> The authors would like to thankfully acknowledge the facilities of Design++ at ETH Zürich and the funding through ETH Foundation grant No. 2020-HS-388 (provided by Kollbrunner/Rodio) as well as the SDSC Project No. C21-08 "Domain-Aware AI-augmented Design of Bridges (DAAAD Bridges)".

\* Corresponding author.

E-mail addresses: [rbischof@ethz.ch](mailto:rbischof@ethz.ch) (R. Bischof), [kraus@ismd.tu-darmstadt.de](mailto:kraus@ismd.tu-darmstadt.de) (M.A. Kraus).

<https://doi.org/10.1016/j.cma.2025.117914>

Received 12 July 2024; Received in revised form 25 November 2024; Accepted 6 March 2025

Available online 16 March 2025

0045-7825/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

design optimisation [10] or structural health monitoring and system identification [11]. PINNs have also been successfully applied in computational fluid mechanics and dynamics for surrogate modelling of numerically expensive fluid flow simulations [12], identification of hidden quantities of interest (velocity, pressure) from spatiotemporal visualisations of a passive scalar (dye or smoke) [13] or in an inverse heat transfer application setting in flow past a cylinder without thermal boundaries [14].

However, further research is necessary to tackle current failure modes [15,16], one of which is the issue of gradient pathologies arising from imbalanced loss terms during training [17]. The various terms in the objective function stemming from physical laws are naturally bound to units of measurement that can vary significantly in magnitude. Consequently, the signal strengths of backpropagated gradients might differ from term to term and lead to pathologies that were shown to impede proper training and cause imbalanced solutions [17], hence posing challenges to global optimisation methods such as Adam, Stochastic Gradient Descent (SGD), or L-BFGS [18–21]. As a countermeasure, every individual term  $i$  may be scaled by a factor  $\lambda_i$  in order to balance its contribution to the total gradient. However, manual tuning of these scaling factors requires laborious grid search and becomes intractable as the number of terms grows.

This work investigates different schemes aiming at adaptively balancing the contributions of multiple terms and their gradients in the loss function by selecting the optimal scaling factors  $\lambda_i$  in order to improve the approximation capabilities of PINNs. To this end, we compare the effectiveness of Learning Rate Annealing (LRAnnealing) [17], proposed in the context of PINNs, to two approaches originating from Computer Vision applications: GradNorm [22] and SoftAdapt [23]. In addition, we derive and present our own variation of an adaptive loss scaling technique, *ReLoBRaLo* (Relative Loss Balancing with Random Lookback), that we found to be more effective at similar efficiency compared to the state-of-the-art by testing the algorithms on various benchmark problems for PINNs in the forward and inverse setting: Helmholtz, Burgers and Kirchhoff PDEs.

This paper is organised as follows: we first provide a short introduction to the problem as well as the state-of-the-art of PINNs in Section 3. Further methodical background on multi-objective optimisation (MOO), the framework of Physics-Informed Neural Networks (PINNs) and loss balancing for PINNs training is presented in Section 5. In Section 6, we introduce *ReLoBRaLo* as a novel self-adaptive loss balancing method. Section 8 reports numerical results of the developed approach against state-of-the-art methods for several examples in the forward and inverse setting: Burgers, Kirchhoff and Helmholtz PDEs. Section 9 presents the results of ablation studies, discusses findings, and draws further conclusions on *ReLoBRaLo* and its hyperparameter settings across all examples of this paper. Finally, a summary and an outlook are given in Section 10. All code produced within this publication is freely available and open access here: [https://github.com/rbischof/relative\\_balancing](https://github.com/rbischof/relative_balancing).

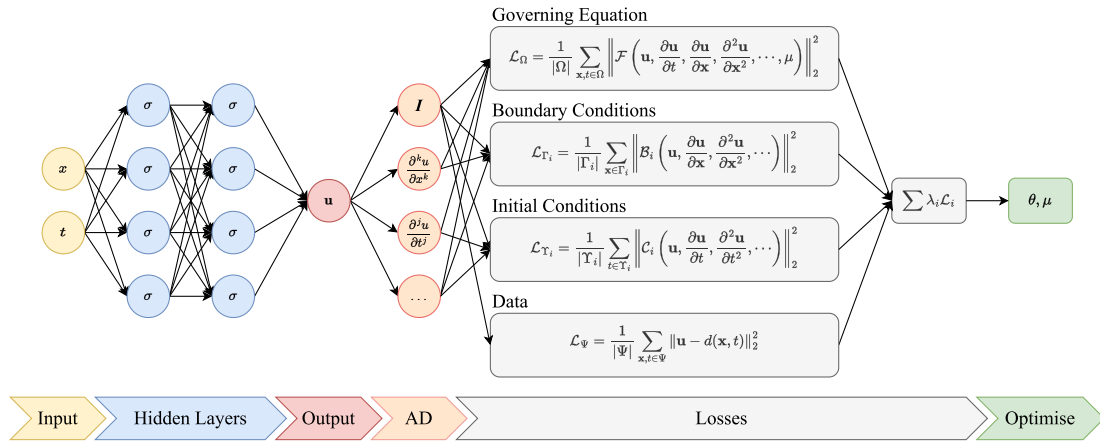
## 2. State-of-the-art and related work

Using neural networks to approximate the solutions of Ordinary Differential Equations (ODEs) and PDEs has been the subject of several studies over the past decade. Initially, Lagaris et al. trained neural networks to solve ODEs and PDEs on a predefined set of grid points [24,25], while Sirignano proposed a method for solving high-dimensional PDEs through approximation of the solution by a neural network and especially emphasising training efficiency by incorporating mini-batch sampling in high dimensional settings compared to the computationally intractable finite mesh-based schemes [26]. Raissi et al. introduced the term Physics-Informed Neural Networks (PINN) and provided empirical justification by numerical simulations for a variety of nonlinear PDEs, including the Navier–Stokes equation and the Burgers equation [27]. Shin et al. provided the first theoretical justification for PINNs by demonstrating the convergence of linear elliptic and parabolic PDEs in the L2 sense [28]. Despite the extensive development of PINN methods for solving Partial Differential Equations (PDEs), a comprehensive comparison across a broad spectrum of PDEs remained absent until the introduction of the PINNacle benchmark [29]. PINNacle is a compilation of benchmarking dataset of numerous distinct PDEs from various domains such as heat conduction, fluid dynamics, biology, and electromagnetics, addressing real-world challenges like complex geometry, multi-scale phenomena, nonlinearity, and high dimensionality.

However, PINN training efficiency, convergence, and accuracy remain serious challenges [12,30]. Current research may be ordered into four main approaches: modifying the structure of the NN, divide-and-conquer/domain decomposition, parameter initialisation and loss balancing.

Jagtap et al. adapted the typical NN architecture to PINNs by introducing parameters that scale the input to the activation functions and get updated alongside the network's parameters  $\theta$  through gradient descent [31,32]. The authors showed that the adaptive activation function significantly accelerated convergence and also improved solution accuracy. Kim et al. presented a fast and accurate PINN ROM with a nonlinear manifold solution representation, where the NN structure included an encoder and a decoder part [33]. Furthermore, a shallow masked encoder was trained using data from the full-order model simulations in order to use the trained decoder as a representation of the nonlinear manifold solution. Peng et al. proposed dictionary-based PINNs to store and retrieve features and speed up convergence by merging prior information into the structure of NNs [34].

Other research focused on decomposing the computational domain in order to accelerate convergence. Jagtap et al. proposed conservative PINNs and extended PINNs [35] that decompose the computational domain into several discrete sub-domains, each one solved independently using a separate, shallow PINN [36,37]. Inspired by this work, Shukla et al. derived and investigated a distributed training framework for PINNs that used domain decomposition methods in both space and time-space [38]. The distributed framework combined the benefits of conservative and extended PINNs to accelerate convergence. The time-space domain may become very large when solving PDEs with long-time integration, causing the training cost of NNs to become extremely expensive. To that end, Meng et al. proposed a parareal PINN to address the long-standing issue [39]. Using a fast coarse-grained solver, the authors decomposed the long-time domain into many discrete short-time domains. Training multiple PINNs with many small data sets was much faster than training a single PINN with a large data set. For PDEs with long-time integration, the parareal



**Fig. 1.** Schematic of a Physics-Informed Neural Network (PINN): A fully-connected feed-forward neural network with space and time coordinates  $(x, t)$  as inputs, approximating a solution  $\hat{u}(x, t)$ . Derivatives of  $u$  w.r.t. inputs are computed by automatic differentiation (AD) and then incorporated into residuals of the governing equations as the loss function, which is composed of multiple terms weighted by different coefficients. Parameters of the FCNN  $\theta$  and the unknown PDE parameters  $\mu$  may be optimised simultaneously by minimising the loss function.

PINN achieved a significant speedup. Kharazmi et al. introduced hp-variational PINNs to divide the computational space into the trial space and test space by combining domain decomposition and projection onto high-order polynomials [40]. A soft split of the problem domain incorporating variants of the Mixture-of-Experts approach was investigated by [41].

In most works, researchers resort to the Xavier initialisation [42] for selecting the PINN's initial weights and biases. The effects of using more refined initialisation procedures have recently been gaining attention, with Liu et al. showing that a good initialisation can provide PINNs with a head start, allowing them to achieve fast convergence and improved accuracy [43]. Transfer learning for PINNs was introduced by Chakraborty et al. [44] and Goswami et al. [45] to initialise PINNs for dealing with multi-fidelity problems and brittle fracture problems, respectively. After their success in other fields of Deep Learning, meta-learning algorithms have also been implemented in the context of PINNs [46–49], with Model-Agnostic Meta-Learning (MAML) being amongst the most popular ones [50]. Its second-order objective is to find a sub-optimal initialisation, from where the network requires only a few labelled training samples and optimisation steps to specialise in a task and achieve high accuracy (few-shot learning). Subsequently, Nichol et al. proposed the REPTILE algorithm, which turns the second-order optimisation of MAML into a first-order approximation, requiring significantly less computation and memory while achieving similar performance [51]. Liu et al. applied the REPTILE algorithm to PINNs by regarding modifications of PDE parameters as separate tasks [43]. The resulting initialisation is such that the PINN converges in just a few optimisation steps for any choice of PDE parameters.

Training PINNs can be particularly challenging due to the absence of prior knowledge about the solutions to PDEs, making the selection of an effective training set difficult. To overcome these challenges, several adaptive sampling strategies have been introduced in the past, cf. the recent review by [52]. While the most common strategy is the residual-based adaptive refinement (RAR) [35], while newer approaches base the adaptive sampling strategy on other measures such as effective failure probabilities [53].

Using derivative information of the target function during training of a neural network was introduced by Czarnecki et al. under the term *Sobolev Training* [54]. Sobolev Training proved to be more efficient in many applicable fields due to lower sample complexity compared to regular training. Son et al. enhance the concept of Sobolev Training in the strict mathematical sense using Sobolev norms in loss functions of neural networks for solving PDEs [55]. It was found that these novel Sobolev loss functions lead to significantly faster convergence on investigated examples compared to traditional L2 loss functions. NNs were used in plain as well as a Sobolev Training manner for constitutive modelling, where it was shown that mechanical relations could be seen as Sobolev training to successfully encapsulate several aspects of the constitutive behaviour, such as strain–stress-relationships arising from derivatives of a Helmholtz potential in hyperelasticity [56–59].

Colby et al. observed that a weighted scalarization of the multiple loss functions, defined by the sampled data and physical laws for PINNs training, plays a significant role in convergence [60]. Wang et al. recently published a Learning Rate Annealing algorithm that employs back-propagated gradient statistics in the training procedure in order to adaptively balance the terms' contributions to the final loss [17] and investigated the issue of vanishing and exploding gradients that currently limits the applicability of PINNs [15]. To that end, the authors introduced a Neural Tangent Kernel (NTK), which appropriately assigns weights to each loss term at subtle performance improvement in order to comprehend the training process for PINNs. Shin et al. developed the Lipschitz regularised loss for solving linear second-order elliptic and parabolic type PDEs [28]. McClenney et al. proposed a method for updating the adaptation weights in the loss function in relation to network parameters [16].

### 3. Physics-Informed Neural Networks (PINNs)

This section reviews basic Physics-Informed Neural Networks (PINNs) concepts and recent developments. Consider the following abstract parameterised and nonlinear PDE problem:

$$\begin{aligned} \text{PDE} : \mathcal{F} \left( \hat{\mathbf{u}}, \frac{\partial \hat{\mathbf{u}}}{\partial t}, \frac{\partial \hat{\mathbf{u}}}{\partial \mathbf{x}}, \dots; \boldsymbol{\mu} \right) &= 0, \quad \mathbf{x} \in \Omega, t \in Y \\ \text{B.C.} : \mathcal{B} \left( \hat{\mathbf{u}}, \frac{\partial \hat{\mathbf{u}}}{\partial \mathbf{x}}, \frac{\partial^2 \hat{\mathbf{u}}}{\partial \mathbf{x}^2}, \dots \right) &= 0, \quad \mathbf{x} \in \Gamma \\ \text{I.C.} : \mathcal{C} \left( \hat{\mathbf{u}}, \frac{\partial \hat{\mathbf{u}}}{\partial t}, \frac{\partial^2 \hat{\mathbf{u}}}{\partial t^2}, \dots \right) &= 0, \quad t \in Y \end{aligned} \quad (1)$$

where  $\mathbf{x} \in \mathbb{R}^d$  is the spatial coordinate and  $t$  is the time;  $\mathcal{F}$  denotes the residual of the PDE, containing the differential operators (i.e.  $\partial_{\mathbf{x}} \hat{\mathbf{u}}, \partial_t \hat{\mathbf{u}}, \dots$ );  $\boldsymbol{\mu} = [\mu_1, \mu_2, \dots]$  are the PDE parameters;  $\hat{\mathbf{u}}(\mathbf{x}, t)$  is the solution of the PDE with initial condition  $\mathcal{C}$  and boundary condition  $\mathcal{B}$  (which can be Dirichlet, Neumann or mixed);  $\Omega$ ,  $\Gamma$  and  $Y$  represent the spatial domain resp. boundary. A special example considered in this paper is the Burgers equation (given in Eq. (12)):  $\partial_t \hat{\mathbf{u}} + \hat{\mathbf{u}} \partial_{\mathbf{x}} \hat{\mathbf{u}} - \nu \partial_{\mathbf{x}}^2 \hat{\mathbf{u}} = 0$  with PDE parameter  $\mu$  as viscosity coefficient  $\nu$ . This paper is concerned with solving forward and inverse problems in different fields of application. For the forward problem, solutions of PDEs are to be inferred with fixed parameters  $\mu$ , while for the inverse problem setting,  $\mu$  is unknown and has to be learned from observed data together with the PDE solution.

Following the “vanilla” implementation of PINNs [1], a fully-connected feed-forward neural network (FCNN)  $U(\mathbf{x}, t; \theta)$  is used to approximate the function  $\hat{\mathbf{u}}(\mathbf{x}, t)$  which solves the PDE. A FCNN consists of multiple hidden layers with trainable parameters (weights and biases; denoted by  $\theta$ ) and takes as inputs the space and time coordinates  $(\mathbf{x}, t)$ , cf. Fig. 1. The losses are then defined as follows:

$$\begin{aligned} \mathcal{L}_{\Omega} &= \frac{1}{|\hat{\Omega}|} \sum_{\mathbf{x}, t \in \hat{\Omega}} \left\| \mathcal{F} \left( \mathbf{u}, \frac{\partial \mathbf{u}}{\partial t}, \frac{\partial \mathbf{u}}{\partial \mathbf{x}}, \frac{\partial^2 \mathbf{u}}{\partial \mathbf{x}^2}, \dots, \boldsymbol{\mu} \right) \right\|_2^2 \\ \mathcal{L}_{\Gamma_i} &= \frac{1}{|\hat{\Gamma}_i|} \sum_{\mathbf{x}, t \in \hat{\Gamma}_i} \left\| \mathcal{B}_i \left( \mathbf{u}, \frac{\partial \mathbf{u}}{\partial \mathbf{x}}, \frac{\partial^2 \mathbf{u}}{\partial \mathbf{x}^2}, \dots \right) \right\|_2^2 \\ \mathcal{L}_{Y_i} &= \frac{1}{|\hat{Y}_i|} \sum_{\mathbf{x}, t \in \hat{Y}_i} \left\| \mathcal{C}_i \left( \mathbf{u}, \frac{\partial \mathbf{u}}{\partial t}, \frac{\partial^2 \mathbf{u}}{\partial t^2}, \dots \right) \right\|_2^2 \\ \mathcal{L}_{\psi} &= \frac{1}{|\hat{\psi}|} \sum_{\mathbf{x}, t \in \hat{\psi}} \|\mathbf{u} - d(\mathbf{x}, t)\|_2^2 \end{aligned} \quad (2)$$

where  $\hat{\Omega}$  is a set of collocation points on the physical domain,  $\hat{\Gamma}_i$  for the boundary conditions (BC),  $\hat{Y}_i$  for the initial conditions (IC) and  $\hat{\psi}$  represents a set of measurements (data); the function  $d$  maps  $(\mathbf{x}, t)$  to measurements at those coordinates;  $\mathbf{u}$  is the output from the neural network  $U(\mathbf{x}, t; \theta)$ . PINNs are generally trained using the L2-norm (mean squared error/MSE) on uniformly sampled collocation points defined as a data set  $\{\mathbf{x}_i, t_i\}_{i=1}^N$  prior to training. Note that the number of points  $N$  (denoted by  $|\cdot|$  in Eq. (2)) may vary for different loss terms.

The objectives in Eq. (2) are trained jointly and hence fall into the class of multi-objective optimisation (MOO) (cf. Eq. (4))

$$\mathcal{L}(\theta, \boldsymbol{\mu}) = (\mathcal{L}_{\Omega}, \mathcal{L}_{\Gamma_1}, \dots, \mathcal{L}_{\Gamma_n}, \mathcal{L}_{Y_1}, \dots, \mathcal{L}_{Y_m}, \mathcal{L}_{\psi})^T \quad (3)$$

where the individual terms can be interpreted in the following way:

- the first term  $\mathcal{L}_{\Omega}$  penalises the residual of the governing equations (PDEs), included in both the forward and inverse problem.
- the following  $n$  terms  $\mathcal{L}_{\Gamma_i}$  enforce the boundary conditions (BCs), included only in the forward problem.
- the following  $m$  terms  $\mathcal{L}_{Y_i}$  enforce the initial conditions (ICs), included only in the forward problem.
- the last term  $\mathcal{L}_{\psi}$  makes the network approximate the measurements, included both in the forward (albeit not strictly necessary) and inverse problem.

A common approach for handling MOO is through linear scalarization, described in more detail in the following Section 4.

### 4. Multi-objective optimisation

Multi-objective optimisation (MOO) is concerned with simultaneously optimising a set of  $k > 1$ , potentially conflicting objectives [61,62].

$$\mathcal{L}(\theta) = (\mathcal{L}_1(\theta), \dots, \mathcal{L}_k(\theta))^T \quad (4)$$

which can be turned into a single objective through linear scalarization:

$$\mathcal{L}(\theta) = \sum_{i=1}^k \lambda_i \mathcal{L}_i(\theta), \quad \lambda_i \in \mathbb{R}_{>0} \quad (5)$$

Many problems in engineering, natural sciences, or economics can be formulated as multi-objective optimisations and generally require trade-offs to simultaneously satisfy all objectives to a certain degree [63]. The solution of MOO models is usually expressed as a set of Pareto optima, representing these optimal trade-offs between given criteria according to the following definitions [64]:

**Definition 4.1.** A solution  $\hat{\theta} \in \Omega$  Pareto dominates solution  $\theta$  (denoted  $\hat{\theta} < \theta$ ) if and only if  $\mathcal{L}_i(\hat{\theta}) \leq \mathcal{L}_i(\theta), \forall i \in \{1, \dots, m\}$  and  $\exists j \in \{1, \dots, m\}$  such that  $\mathcal{L}_j(\hat{\theta}) < \mathcal{L}_j(\theta)$ .

**Definition 4.2.** A solution  $\hat{\theta} \in \Omega$  is said to be Pareto optimal if  $\forall \theta \in \Omega, \hat{\theta} \leq \theta$ . The set of all Pareto optimal points is called the Pareto set, and the image of the Pareto set in the loss space is called the Pareto front.

In theory, a Pareto optimal solution  $\theta$  is independent of the scalarisation [65]. However, the solution space becomes highly non-convex when using neural networks for MOO. Thus, although neural networks are universal function approximators [66], they are not guaranteed to find the globally optimal solution through gradient-based optimisation. Scaling the loss space, therefore, provides the option of guiding the gradients into having an a priori deemed desirable property. However, manually finding optimal  $\lambda_i$  requires laborious grid search and becomes intractable as  $k$  gets large. Furthermore, one might want to let  $\lambda_i$  evolve over time. This raises the need for an automated scheme to dynamically choose the scalings  $\lambda_i$ .

## 5. Adaptive loss balancing schemes

This section reviews different methods aiming at balancing the various terms within multi-objective optimisation. To this end, we compare the effectiveness of Learning Rate Annealing [17], proposed in the context of PINNs, as well as two approaches originating from Computer Vision applications: GradNorm [22] and SoftAdapt [23]. This forms the basis for deriving and presenting our own loss balancing method as given in Section 6.

### 5.1. Learning rate annealing

Wang et al. [17] conducted a study on gradients in PINNs and identified pathologies that explained some failure modes. One pathology is gradient stiffness in the boundary conditions caused by the imbalance amongst the different loss terms. As a remedy, it is proposed to adaptively scale the loss using gradient statistics, thus reducing the laborious tuning of these hyperparameters.

$$\begin{aligned}\hat{\lambda}_i(t) &= \frac{\max\{|\nabla_{\theta} \mathcal{L}_{\Omega}(t)|\}}{|\nabla_{\theta} \mathcal{L}_{\{I,Y\}}(t)|}, \quad i \in \{1, \dots, k\} \\ \lambda_i(t) &= \alpha \lambda_i(t-1) + (1-\alpha) \hat{\lambda}_i(t) \\ \theta^{(t+1)} &= \theta^{(t)} - \eta \nabla_{\theta} \left( \mathcal{L}_{\Omega}(t) + \sum_{i=1}^k \lambda_i(t) \mathcal{L}_{\{I,Y\}}(t) \right)\end{aligned}\tag{6}$$

where  $\overline{|\nabla_{\theta} \mathcal{L}_{\{I,Y\}}(t)|}$  is the mean of the gradient w.r.t. the parameters  $\theta$ ;  $\alpha$  is a hyperparameter with a value  $\alpha = 0.9$  recommended by the authors.

With this method, whenever the maximum value of  $|\nabla_{\theta} \mathcal{L}_{\Omega}(t)|$  grows considerably larger than the average value in  $|\nabla_{\theta} \mathcal{L}_{\{I,Y\}}(t)|$ , the scalings  $\hat{\lambda}_i(t)$  correct for this discrepancy such that all gradients have similar magnitudes. Additionally, exponential decay is used in order to smoothen the balancing and avoid drastic changes of the loss space between optimisation steps.

This procedure induces a few drawbacks. Its unboundedness potentially involves up- or down-scaling of terms by means of several orders of magnitude. Up-scaling, in particular, can cause problems similar to the effect of choosing a learning rate that is too large and, therefore, leads to repeatedly overshooting the objective. Furthermore, scaling all terms to have the same magnitude throughout training can incite the network to optimise for the “low-hanging fruit”. A term whose loss decreased considerably in the last optimisation step will see its contribution to the total gradient scaled back up to the same magnitude to match the other terms. Therefore, the network might focus on the objectives that are easiest to optimise for.

### 5.2. GradNorm

Chen et al. [22] take a different approach, making the scalings  $\lambda_i$  trainable. The updates on these trainable scalings are chosen such that all terms improve at the same relative rate w.r.t. their initial loss and are performed by a separate optimiser. A term that has improved at a higher rate since the beginning of training compared to the other terms gets a weaker scaling until all terms have made the same relative progress. Therefore, one could argue that they weakly enforce each optimisation step to Pareto dominate (cf. Definition 4.1) its predecessor. The loss for updating the scalings within GradNorm is computed as follows:

$$\mathcal{L}(t; \lambda) = \sum_{i=1}^k \left| G_{\theta}^{(i)}(t) - \overline{G_{\theta}}(t) \times [r_i(t)]^{\alpha} \right|_1\tag{7}$$

where  $G_{\theta}^{(i)}(t) = \|\nabla_{\theta} \lambda_i \mathcal{L}_i(t)\|_2$  is the  $L_2$  norm of the gradient w.r.t. the network parameters  $\theta$  for the scaled loss of objective  $i \in \{1, \dots, k\}$ ;  $\overline{G_{\theta}}(t) = \frac{1}{k} \sum_{i=1}^k G_{\theta}^{(i)}(t)$  is the average of all gradient norms;  $r_i(t) = \mathcal{L}_i(t)/\mathcal{L}_i(0)$  defines the rate at which term  $i$  improved

so far;  $\alpha$  is a hyperparameter representing the strength of the restoring force which pulls tasks back to a common training rate. Note that  $\bar{G}_\theta(t) \times [r_i(t)]^\alpha$  is the desirable value that  $G_\theta^{(i)}(t)$  should take on, so gradients must be prevented from flowing through this expression. The final loss for updating the network parameters is then simply a linear scalarization with the scalings that were previously updated:

$$\mathcal{L}(t; \theta) = \sum_{i=1}^k \lambda_i(t) \mathcal{L}_i(t) \quad (8)$$

This algorithm is fairly evolved, and despite solving some of Learning Rate Annealing's issues, it still requires a separate backward-pass for each task, which becomes prohibitively expensive as  $k$  gets large. Furthermore, it relies on two separate optimisation rounds at each step: one for adapting the scalings  $\lambda_i$  and another for updating the weights  $\theta$ . By means of Eq. (4), GradNorm can thus be formulated as a scalarised MOO via:

$$\mathcal{L}(t) = (\mathcal{L}(t; \theta), \mathcal{L}(t; \lambda))^T \quad (9)$$

This in turn requires empirical hyperparameter tuning (learning rate, initialisation, etc.) to keep the system balanced — exactly the problem we are trying to solve through adaptive loss balancing schemes.

### 5.3. SoftAdapt

Like GradNorm, SoftAdapt [23] leverages the ansatz of relative progress to balance the loss terms. However, the authors relax it by only considering the previous time-step  $\mathcal{L}_i(t-1)$  and taking the difference between time steps instead of the division. The scalings are then normalised by using a softmax function:

$$\lambda_i(t) = \frac{\exp(\mathcal{T}(\mathcal{L}_i(t) - \mathcal{L}_i(t-1)))}{\sum_{j=1}^k \exp(\mathcal{T}(\mathcal{L}_j(t) - \mathcal{L}_j(t-1)))}, \quad i \in \{1, \dots, k\} \quad (10)$$

where  $\mathcal{L}_i$  is the loss of term  $i$  at optimisation step  $t$  and  $\mathcal{T}$  is a tunable hyper-parameter that influences the weighting of different components within the loss function based on their performance. Specifically, setting  $\mathcal{T} > 0$  causes the algorithm to allocate more weight to the worst performing component, identified by the most positive rate of change in the loss. Conversely, choosing  $\mathcal{T} < 0$  prioritises the best performing components, those with the most negative rate of change. Setting  $\mathcal{T} = 0$  results in equal weighting across all components, disregarding their individual performance changes.

SoftAdapt also differs from GradNorm in the sense that it does not require gradient statistics and thus eliminates the need to perform separate backward passes for each objective. Instead, it makes use of the fact that magnitudes in the gradients directly depend on the magnitudes of the terms in the loss function and, therefore, aims at achieving the balance solely through loss statistics. This is obviously true only if the same loss function is used for every objective (e.g. the  $L_2$  loss). However, this setting generalises to a vast majority of applications involving PINNs.

## 6. Relative Loss Balancing with Random Lookback (ReLoBRaLo)

Drawing inspiration from existing balancing techniques as outlined in Section 5, we propose a novel method and implementation for balancing the multiple terms in the scalarised MOO loss function for training of PINNs upon:

- SoftAdapt's concept of operating on loss statistics as opposed to gradient statistics is employed. A computationally inexpensive softmax ensures the sum of scalings is bounded.
- Inspired by GradNorm, the progress is calculated by dividing the loss at the current iteration  $\mathcal{L}_i(t)$  by the loss at the previous iteration  $\mathcal{L}_i(t-1)$ .
- Similarly to Learning Rate Annealing, the scalings are updated using an exponential decay in order to utilise loss statistics from more than just one training step in the past.
- In addition, a random lookback (called *saudade*  $\rho$ ) is introduced into the exponential decay, which decides whether to use the previous steps' loss statistics to compute the scalings or whether to look all the way back until the start of training  $\mathcal{L}_i^{(0)}$ .

$$\begin{aligned} \lambda_i^{bal}(t, t') &= m \cdot \frac{\exp\left(\frac{\mathcal{L}_i(t)}{\mathcal{T}\mathcal{L}_i(t')}\right)}{\sum_{j=1}^m \exp\left(\frac{\mathcal{L}_j(t)}{\mathcal{T}\mathcal{L}_j(t')}\right)}, \quad i \in \{1, \dots, m\} \\ \lambda_i^{hist}(t) &= \rho \lambda_i(t-1) + (1-\rho) \lambda_i^{bal}(t, 0) \\ \lambda_i(t) &= \alpha \lambda_i^{hist} + (1-\alpha) \lambda_i^{bal}(t, t-1) \end{aligned} \quad (11)$$

where  $\alpha$  is the exponential decay rate,  $\rho$  is a Bernoulli random variable and  $\mathbb{E}[\rho]$  should be chosen close to 1. The intermediate step  $\lambda_i^{bal}(t, t')$  calculates scalings based on the relative improvements of each term between time steps  $t'$  and  $t$ . The following step  $\lambda_i^{hist}(t)$  defines whether the scalings calculated in the previous time step ( $\rho$  evaluates to 1) or the relative improvements since the beginning of training ( $\rho$  evaluates to 0) should be carried forward. Note that this concept of randomly retaining or discarding the history of

**Table 1**

Hyperparameters for architecture and training settings together with ranges as used for Bayesian Optimisation.

| Hyperparameter                      | Range                       | Log-scaling |
|-------------------------------------|-----------------------------|-------------|
| Learning Rate $l_r$                 | $[10^{-6}, 10^{-2}]$        | Yes         |
| Layers $d_K$                        | $[2, 4]$                    | No          |
| Neurons per Layer $w_K$             | $[32, 512]$                 | No          |
| Exponential Decay Rate $\alpha$     | $[0, 1]$                    | No          |
| Temperature $\mathcal{T} = 10^i$    | $[10^{-6}, 10^2]$           | Yes         |
| Expected Saudade $\mathbb{E}[\rho]$ | $[0, 1]$                    | No          |
| Activation function $\sigma$        | $\{\tanh, \text{sigmoid}\}$ | No          |

scalings is what we denote as “random lookbacks”. Finally, the scaling  $\lambda_i(t)$  for term  $i$  is obtained by means of an exponential decay, where  $\alpha$  controls the weight given to past scalings versus the scalings calculated in the current time step.

This method attempts to combine the best attributes of the aforementioned approaches into a new scheme for scalarised MOO objective functions. First and foremost, it still weakly enforces every training step to Pareto dominate its predecessor, which is an important property in physical applications. It also avoids using gradient statistics, making it considerably more efficient than Learning Rate Annealing and GradNorm. Furthermore, it reduces drastic changes in the loss space by using exponential decay and can easily be adapted to use more or fewer information of past optimisation steps by tuning the hyperparameter  $\alpha$ . One can think of  $\alpha$  as the model’s ability to remember the past, with a high alpha giving lots of weight to past loss statistics, while a lower alpha increases stochasticity. Setting  $\alpha = 1$  results in each term’s relative progress being computed w.r.t. the initial loss  $\mathcal{L}_i^{(0)}$ . However, we found this to be too restrictive since it causes the model to stop making progress as soon as one term reaches a local minimum. We chose values  $\alpha$  between 0.9 and 0.999 and report the effects of varying this hyperparameter in Section 9.

Choosing the value of  $\alpha$  also requires a trade-off: a high value means the model will remember the potential deterioration of certain terms for longer and, therefore, leave a longer time frame to compensate them. However, it also induces a latency between a term starting to deteriorate and the scalings  $\lambda_i$  reacting accordingly. We therefore study the effect of introducing the saudade Bernoulli random variable  $\rho$  that causes the model to occasionally look back until the start of training.  $\mathbb{E}[\rho] = 0$  is maximum saudade as it always takes the loss value of the initial training step, while  $\mathbb{E}[\rho] = 1$  corresponds to minimum saudade, taking only into account the last value from the history of the  $i$ th scaling factor. Selecting  $\mathbb{E}[\rho]$  somewhere between 0 and 1 allows to set a lower value for  $\alpha$ , thus making the model more flexible while still occasionally “reminding” it of the progress made since the start of training. Furthermore, the random lookback can give episodic new impulses and let the model escape local minima by changing the loss space and inciting it to explore more of the parameter space. In case the impulse turns out to have a negative effect on the accuracy, one can still choose to roll back and reset the network’s parameters  $\theta$  to the previous state.

The last hyperparameter is the so-called temperature  $\mathcal{T}$ . Setting  $\mathcal{T} \rightarrow \infty$  re-calibrates the softmax to output uniform values and thus all  $\hat{\lambda}_i^{(t)} = 1$ . On the other hand,  $\mathcal{T} \rightarrow 0$  essentially turns the softmax into an argmax function, with the scaling  $\hat{\lambda}_i^{(t)} = k$  resulting for the term with the lowest relative progress and  $\hat{\lambda}_i^{(t)} = 0$  for all others. A pedagogical example with the interpretation of expected behaviours and how to draw conclusions from the histories of the scalings are given for Burgers’ equation in Section 8.1.

Note that the network should be prevented from optimising this expression. This can be achieved by stopping the gradients from flowing through the calculation of the scalings. Also, depending on the problem at hand,  $\exp(\mathcal{L}_i(t)/(\mathcal{T}\mathcal{L}_i(t')))$  could evaluate to a very large number, thus leading to overflows. This issue can be preemptively tackled by subtracting a large number (e.g.  $10^{-9}$ ) from the input to the softmax.

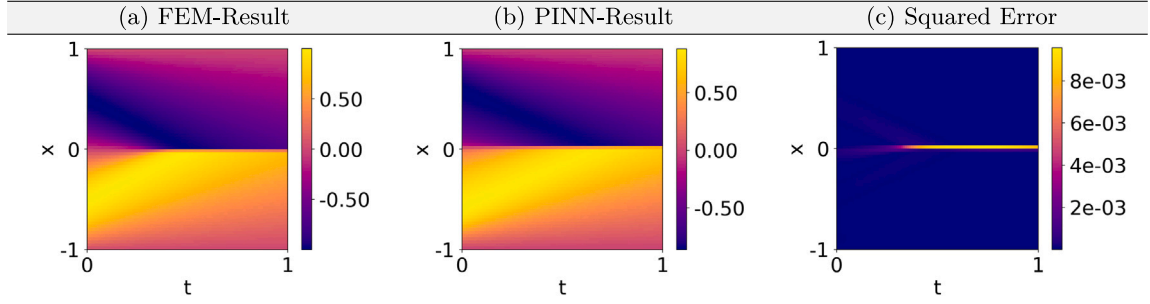
## 7. Hyperparameter tuning and meta learning

We combined grid search with Bayesian Optimisation (BO) [67–69] for hyperparameter tuning. This study uses hyperparameters for defining the NN architecture ( $d_K$  hidden layers and  $w_K$  neurons per layer) and training settings (learning rate  $l_r$ , exponential decay rate  $\alpha$  and saudade  $\rho$ ). Table 1 contains the ranges and distributions for the hyperparameters. Bayesian Optimisation reduces the empiricism of selecting the PINN’s hyperparameters to learn an optimal NN structure. First, 20 random points in the hyperparameter space are sampled and evaluated. The model’s performance at those points serves as evidence for fitting prior Gaussian Processes in order to estimate the unknown loss function w.r.t. the hyperparameters. Using Expected Improvement (EI) [70], a further 80 points are then sampled and evaluated to refine the prior. This procedure provides an educated guess as to which are the optimal hyperparameters for the task at hand. Finally, we fine-tune the results by performing a fine-grained grid search around the hyperparameters returned by Bayesian Optimisation (BO).

Within this study, the exact same Bayesian Optimisation configuration was used for all examples presented in Section 8. Hence, it is sufficient to only display Table 1. Respective results of the Grid Search and Bayesian Optimisation can also be found in Section 8.

## 8. Results

We evaluate the different balancing schemes on the PINNacle benchmark as well as on additional PDE problems (Burgers’ equation, Kirchhoff plate bending, Helmholtz equation, and Allen-Cahn equation) originating from physics-informed deep learning, where the objective function consists of various terms of potentially considerably different magnitudes and compare their performances, as well as their computational efficiency. The training was done on networks of varying depth and width (acc. to Table 1) and limited



**Fig. 2.** Burgers' equation problem: (a) FEM reference solution, (b) PINNs results predicted with a fully-connected network consisting of two layers and 128 nodes each, and (c) squared error.

to  $10^5$  steps of gradient descent (GD) using the Adam optimiser [19]. Additionally, we reduced the learning rate by a multiplicative factor of 0.1 whenever the optimisation stopped making progress for over 3'000 optimisation steps and finally used early stopping in case of 9'000 steps without improvement. When addressing the inverse problem, i.e. approximating a set of measurements while subjecting the network to PDE constraints for finding unknown PDE parameters  $\mu$ , we further investigated the payoff of using two separate optimisers: one for updating network weights  $\theta$  and a separate one for updating PDE parameters  $\mu$ . Further details on hyperparameter tuning and meta-learning are given in Section 9.

### 8.1. Burgers' equation

Burgers' equation is a one-dimensional PDE describing the main properties of the Navier–Stokes equations [71] used, i.a., to model shock waves, gas dynamics, or traffic flow [72]. Using Dirichlet boundary conditions, the PDE takes the following form:

$$\begin{aligned} \frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2} &= 0, \quad x \in [-1, 1], \quad t \in [0, 1] \\ u(0, x) &= -\sin(\pi x) \\ u(t, -1) &= u(t, 1) = 0 \end{aligned} \quad (12)$$

At first, we investigate the solution of the forward problem, where we set the PDE parameter  $\mu := \nu = \frac{1}{100\pi}$ . In order to find the latent function  $\hat{u}(\mathbf{x}, t)$ , we can parameterise it with a neural network  $U(x, t; \theta)$  and turn the set of equations into a linear scalarised objective (cf. Eq. (5)) of Mean Squared Errors (MSE). This loss function will weakly enforce the network to approximate the PDE solution  $\hat{u}(\mathbf{x}, t)$ .

$$\begin{aligned} \mathcal{L}_\Omega &= \frac{1}{|\hat{\Omega}|} \sum_{(x,t) \in \hat{\Omega}} \left\| \frac{\partial U}{\partial t} + U \frac{\partial U}{\partial x} - \nu \frac{\partial^2 U}{\partial x^2} \right\|_2^2 \\ \mathcal{L}_{I_1} &= \frac{1}{|\hat{I}_1|} \sum_{t \in \hat{I}_1} \|U(-1, t; \theta)\|_2^2 \\ \mathcal{L}_{I_2} &= \frac{1}{|\hat{I}_2|} \sum_{t \in \hat{I}_2} \|U(1, t; \theta)\|_2^2 \\ \mathcal{L}_Y &= \frac{1}{|\hat{Y}|} \sum_{x \in \hat{Y}} \|U(x, 0; \theta) + \sin(\pi x)\|_2^2 \end{aligned} \quad (13)$$

For the forward problem, PINN training induces the following loss function employed during training:

$$\mathcal{L} = \lambda_0 \mathcal{L}_\Omega + \lambda_1 \mathcal{L}_{I_1} + \lambda_2 \mathcal{L}_{I_2} + \lambda_3 \mathcal{L}_Y \quad (14)$$

After a successful convergence of the PINNs training using ReLoBRaLo, we obtain the results displayed in Fig. 2(b), whereas the final algorithm settings are reported in Table 8. As there is no analytical solution available for the Burgers equation, we compared the results to a reference solution calculated using the finite element method (FEM), displayed in Fig. 2(a). A plot of the squared difference in  $u$  as given by the FEM and PINNs is shown in Fig. 2(c) and delivers a relative max error of below 5%.

However, Burgers' equation can also be turned into an *inverse problem* by regarding the PDE parameter  $\nu$  as an unknown to be estimated from a set of observations (i.e. data) over the spatial and temporal domain. In this setting, the PINNs induced loss function to be deployed reads:

$$\begin{aligned} \mathcal{L} &= \lambda_0 \frac{1}{|\hat{\Omega}|} \sum_{(x,t) \in \hat{\Omega}} \left\| \frac{\partial U}{\partial t} + U \frac{\partial U}{\partial x} - \nu \frac{\partial^2 U}{\partial x^2} \right\|_2^2 \\ &\quad + \lambda_1 \frac{1}{|\hat{Y}|} \sum_{(x,t) \in \hat{Y}} \|U - u\|_2^2 \end{aligned} \quad (15)$$

**Table 2**

Comparison of the median  $L_2$  training and validation loss on Burgers' equation against a baseline of manually chosen scalings. The reported values are the median over four independent runs with identical settings. Additionally, we report the standard deviation over the runs of the best-performing model on the validation loss.

| Burgers |                                    | Baseline             | GradNorm            | LR anneal.           | SoftAdapt           | ReLoBRaLo            |
|---------|------------------------------------|----------------------|---------------------|----------------------|---------------------|----------------------|
| Forward | train $f$                          | $5.5 \cdot 10^{-4}$  | $6.6 \cdot 10^{-4}$ | $9.9 \cdot 10^{-4}$  | $2.0 \cdot 10^{-4}$ | $5.6 \cdot 10^{-5}$  |
|         | val $u$                            | $1.2 \cdot 10^{-3}$  | $2.0 \cdot 10^{-3}$ | $1.6 \cdot 10^{-4}$  | $8.1 \cdot 10^{-4}$ | $1.4 \cdot 10^{-4}$  |
|         | std val $u$                        | $5.7 \cdot 10^{-4}$  | $2.1 \cdot 10^{-3}$ | $2.3 \cdot 10^{-4}$  | $9.5 \cdot 10^{-3}$ | $6.8 \cdot 10^{-4}$  |
| Inverse | val $\mu$                          | $1.9 \cdot 10^{-10}$ | $6.8 \cdot 10^{-5}$ | $2.5 \cdot 10^{-11}$ | $1.1 \cdot 10^{-7}$ | $2.2 \cdot 10^{-10}$ |
|         | $\nu = \frac{1}{100\pi}$ std $\mu$ | $1.2 \cdot 10^{-9}$  | $5.1 \cdot 10^{-5}$ | $3.4 \cdot 10^{-11}$ | $5.7 \cdot 10^{-7}$ | $2.1 \cdot 10^{-10}$ |

**Table 3**

Median computational overhead  $\Delta T_{co}$  (in s) per 1'000 optimisation steps compared to using no balancing scheme (3.7 s) on Burgers' equation.

|                     | GradNorm | LR ann. | SoftAdapt | ReLoBRaLo |
|---------------------|----------|---------|-----------|-----------|
| $\Delta T_{co}$ [s] | +10.6    | +3.5    | +0.4      | +0.6      |

Similar to the network's weights and biases, the additional trainable PDE variable  $\mu$  (here viscosity  $\nu$ ) is now also updated through gradient descent:

$$\begin{aligned}\theta^{(t)} &= \theta^{(t-1)} - \eta \nabla_{\theta} \mathcal{L} \\ \mu^{(t)} &= \mu^{(t-1)} - \eta \nabla_{\mu} \mathcal{L}\end{aligned}\tag{16}$$

Measurement data  $\mathcal{V}$  for the inverse problem setting was obtained from our reference solution computed using the FEM without the addition of noise. At every iteration, we sample from the available data in order to generate a batch of collocation points.

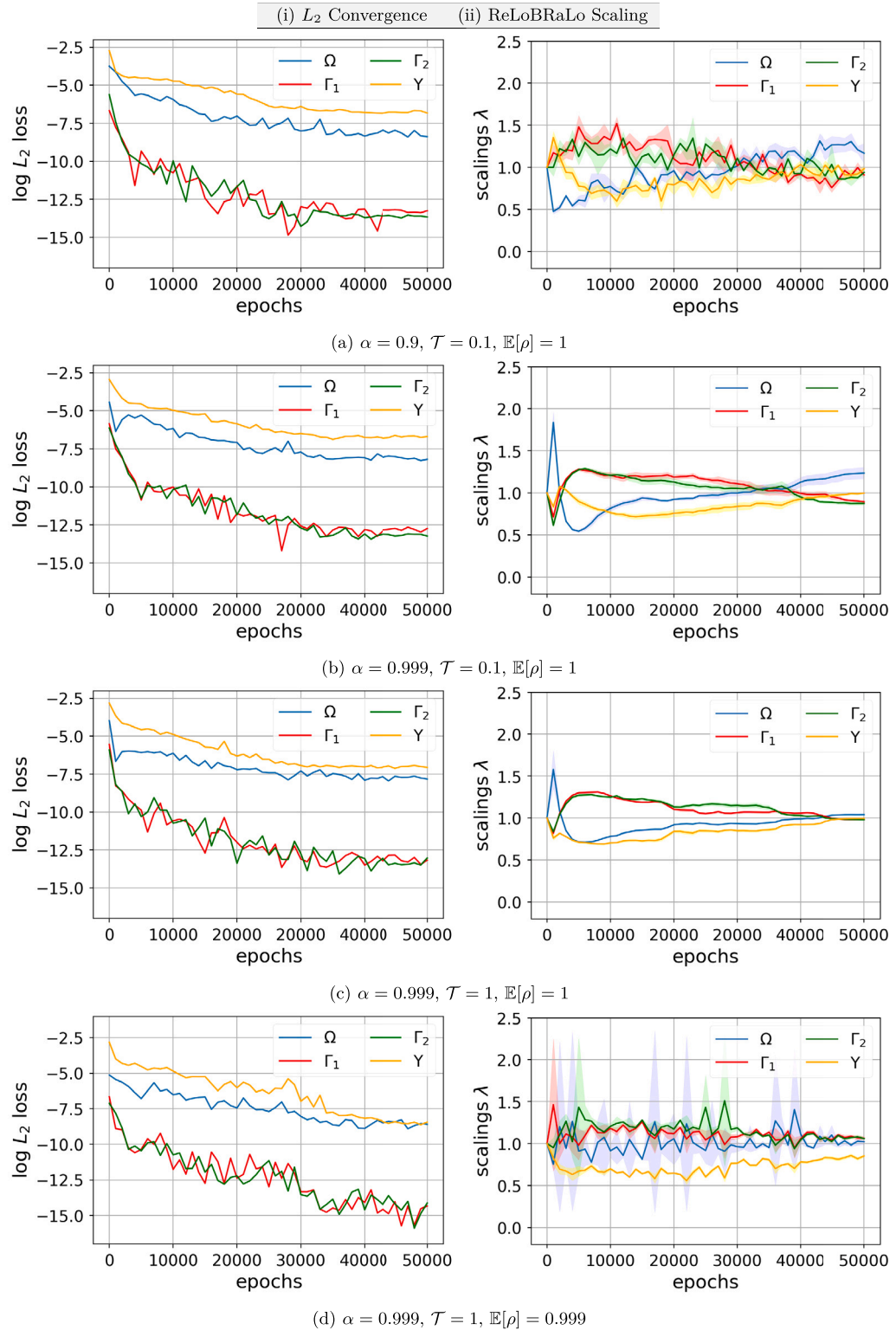
Fig. 3 shows the scaling factors  $\lambda_i$  of our ReLoBRaLo method with varying hyperparameters for Burgers' equation. As can be expected, a larger value for  $\alpha$  leads to a smoother curve because past loss statistics are dragged on longer, countering the stochasticity that might arise at every optimisation step. On the other hand, the temperature  $\mathcal{T}$  influences the magnitude of the scalings. Another general tendency in the plots of the scaling factors is the fact that the relatively lower loss contributions (here: BC 1 and BC 2) correspond to higher scaling values (potentially greater than 1), while the larger loss contributions (here: PDE and IC) correspond to lower scaling values (potentially less than 1). Note that whenever “log” is used in this and all subsequent figures, we refer to the natural logarithm of that quantity.

The relatively small variances across training runs with  $\mathbb{E}[\rho] = 1$  suggest that the optimisation progress follows similar patterns, even when varying the depth and width of the network. Therefore, these values can provide valuable insight into the training and help to identify possibilities for improving the model. For example, the fact that the scaling for the governing equation has the largest value after 50,000 epochs indicates that it was the first term to stop making progress. We will see in the following sections that the opposite holds true for Helmholtz' and Kirchhoff's equations, where the boundary conditions have more difficulties making progress (cf. Fig. 6). This knowledge can help make informed decisions to improve the framework, e.g. by adapting the activation functions, the loss function or the model's architecture accordingly.

Table 2 summarises the performances of the different balancing techniques against a baseline for the forward and inverse problem setting, where we manually chose the optimal scalings  $\lambda_i$  through grid search. As can be observed, the adaptive scaling techniques perform similarly well to the baseline, with Learning Rate Annealing and ReLoBRaLo reaching a considerably lower validation error. The results show that either one of these methods greatly reduces the amount of work required for hyperparameter search while still achieving great results with high probability.

Besides accuracy, computational efficiency is another important metric for evaluating adaptive loss balancing methods. By designing our ReLoBRaLo method requiring only one backward pass, its computational overhead can be expected to be relatively small compared to GradNorm and Learning Rate Annealing, which both utilise gradient statistics and separate backward passes for each term. Indeed, Table 3 shows that Burgers' equation with its four terms in the loss function can be solved by ReLoBRaLo about 40% faster than Learning Rate Annealing and 70% faster than GradNorm and thus adds to efficiency and sustainability of PINNs training. Note that the reported values in Table 3 stem from tasks where the balancing operation was performed at every optimisation step. Both GradNorm and Learning Rate Annealing can be made more efficient by updating the scaling terms once every arbitrary number of iterations. However, this introduces a trade-off between flexibility and efficiency and, therefore, an additional, very sensitive hyperparameter with a high impact on the method's accuracy and efficiency. On the other hand, ReLoBRaLo adapts its scalings at every iteration and has a very low computational cost.

Notably, while the forward problem induced a loss function consisting of four terms, the inverse problem requires only two terms (Eq. (15)). Hence, selecting the scalings manually is significantly less time-consuming than it is for the forward problem. Consequently, Table 2 shows that the baseline was harder to outperform, with Learning Rate Annealing (LR Annealing) and ReLoBRaLo being the only methods yielding better results. It is worth noting, however, that Learning Rate Annealing approximates the true value of  $\nu$  significantly faster than ReLoBRaLo and is, therefore, the optimal choice for this particular problem setting, cf. Fig. 4. Further conclusions and comparisons are made across different loss balancing methods in Section 9.



**Fig. 3.** Median of the  $\log L_2$  loss over multiple training runs (i/left) of Burgers' equation and the mean and variance of the corresponding scaling factors  $\lambda$  (ii/right) computed with ReLoBRaLo.

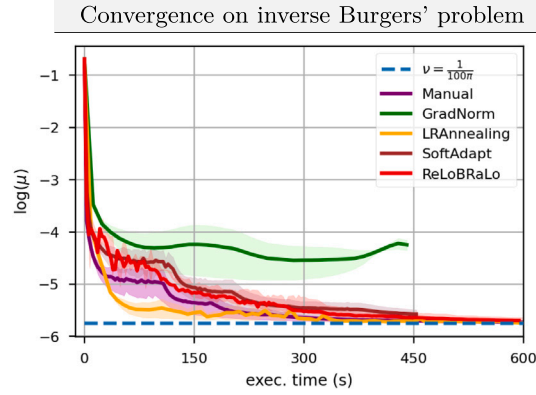


Fig. 4. Approximation of the true PDE parameter value  $\nu$  (dashed line) for the inverse problem setting of Burgers' equation. Reported values are the mean (solid line) and standard deviation (shaded area) of four independent computational runs.

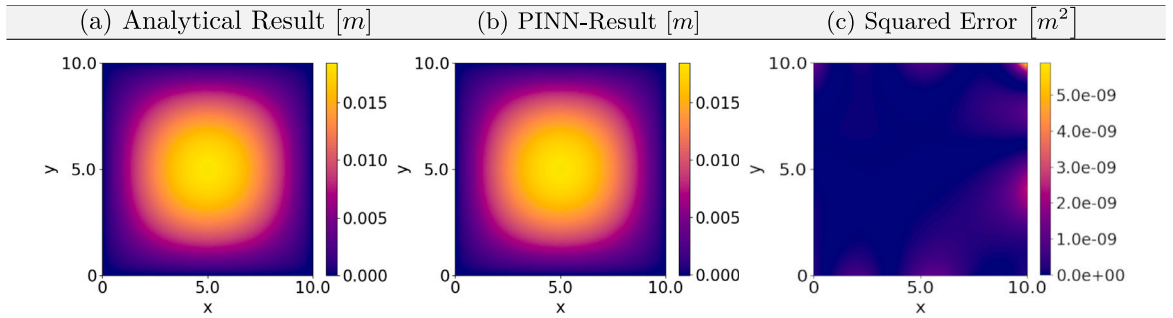


Fig. 5. Kirchhoff plate bending problem: (a) analytical reference solution [m], (b) PINNs results [m] predicted with a fully-connected network consisting of three layers and 128 nodes each, and (c) squared error [m<sup>2</sup>].

## 8.2. Kirchhoff plate bending equation

The Kirchhoff–Love theory of plates arose from civil and mechanical engineering and consists of a two-dimensional mathematical model used to determine stresses and deformations in thin plates subjected to forces and moments [3]. The Kirchhoff plate bending problem assumes that a mid-surface plane can be used to represent a three-dimensional plate in two-dimensional form, and together with a linear elastic material, a fourth-order PDE can be derived to describe its mechanical behaviour:

$$\nabla^4 u(x, y) - \frac{p(x, y)}{D} = 0, \quad (x, y) \in \mathbb{R}_{>0}^2 \quad (17)$$

$$D = \frac{Eh^3}{12(1 - \nu^2)}$$

where  $p(x, y)$  is the load acting on the plate at coordinates  $(x, y)$ ;  $D$  is the plate's flexural stiffness computed with Young's modulus  $E$ , the plate's thickness  $h$  and Poisson's ratio  $\nu$ . The Kirchhoff plate bending problem poses several severe problems to FEM solutions [3], yet analytical solutions can be inferred e.g. using Fourier series for special cases such as an applied sinusoidal load:

$$p(x, y) = p_0 \sin\left(\frac{x\pi}{a}\right) \sin\left(\frac{y\pi}{b}\right)$$

$$u(x, y) = \frac{p_0}{\pi^4 D \left(\frac{1}{a^2} + \frac{1}{b^2}\right)^2} \sin\left(\frac{x\pi}{a}\right) \sin\left(\frac{y\pi}{b}\right) \quad (18)$$

In this paper, we consider a concrete plate of width  $a = 10$  m, length  $b = 10$  m, base load  $p_0 = 0.015$  MN m<sup>-2</sup>, Young's modulus  $E = 30.000$  MN m<sup>-2</sup>, plate height  $h = 0.2$  m and Poisson's ratio of  $\nu = 0.2$ , as well as simply supported edge boundary conditions as it arises in typical civil engineering structures such as slabs [73]. We hence consider the following boundary conditions (BC):

$$u(0, y) = u(a, y) = u(x, 0) = u(x, b) = 0$$

$$m_x(0, y) = m_x(a, y) = m_y(x, 0) = m_y(x, b) = 0 \quad (19)$$

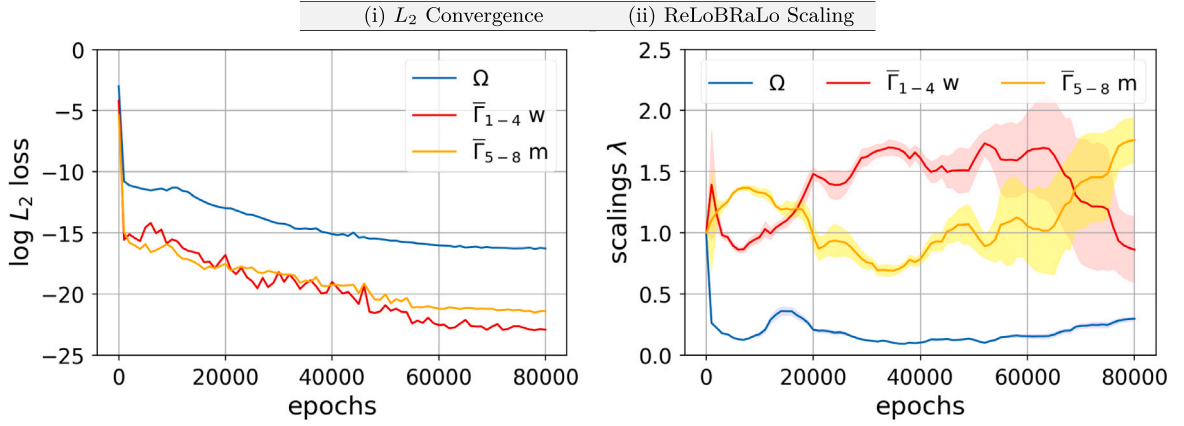


Fig. 6. Median of the log  $L_2$  loss over multiple training runs (a) and the mean and variance of the corresponding scaling factors  $\lambda$  (b) computed with ReLoBRaLo on Kirchhoff's equation with  $\alpha = 0.999$ ,  $\tau = 0.1$ ,  $\mathbb{E}[\rho] = 1$ . For the sake of readability, the boundary conditions 1–4 and 5–8 were aggregated by taking the mean value. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

where  $m_x$  and  $m_y$  are bending moments computed as follows:

$$\begin{aligned} m_x(x, y) &= -D \left( \partial_x^2 u + \nu \partial_y^2 u \right) \\ m_y(x, y) &= -D \left( \nu \partial_x^2 u + \partial_y^2 u \right) \\ m_{xy}(x, y) &= -D(1 - \nu) \partial_{xy}^2 u \end{aligned} \quad (20)$$

In total, we obtain 8 boundary conditions and, therefore 9 terms in the PINNs loss function, making this a challenging task for balancing the contributions of the various objectives:

$$\begin{aligned} \mathcal{L}^{(t)} &= \frac{\lambda_0}{|\hat{\Omega}|} \sum_{x,y \in \hat{\Omega}} \left\| \nabla^4 U(x, y; \theta) - \frac{p}{D} \right\|_2^2 \\ &+ \sum_{i=1}^4 \frac{\lambda_i}{|\hat{\Gamma}_i|} \sum_{x,y \in \hat{\Gamma}_i} \|U(x, y; \theta)\|_2^2 \\ &+ \sum_{i=5}^6 \frac{\lambda_i}{|\hat{\Gamma}_i|} \sum_{x,y \in \hat{\Gamma}_i} \|m_x(x, y)\|_2^2 \\ &+ \sum_{i=7}^8 \frac{\lambda_i}{|\hat{\Gamma}_i|} \sum_{x,y \in \hat{\Gamma}_i} \|m_y(x, y)\|_2^2 \end{aligned} \quad (21)$$

After the successful convergence of the PINNs training, we obtain the results displayed in Fig. 5(b) and compare it to the analytically available solution displayed in Fig. 5(a). A plot of the squared difference in  $u$  as given by the analytical and PINNs results is shown in Fig. 5(c) and delivers a negligible maximum error. The final algorithm settings are reported in Table 8.

Fig. 6 shows an example of ReLoBRaLo's training progress on Kirchhoff's equation. In this particular example, one can notice the larger variance of scaling values towards the end of training. Also, the scalings did not converge towards the value 1, thus suggesting that the training finished without all terms having stopped making progress, i.e. the scalings for the boundary conditions on the moments (yellow) were increasing at the end of training, while the boundary conditions on the displacements (red) were decreasing. This gives a strong indication as to where the model's limitations lie. In this case, additional attention should be paid to the moments, e.g. by selecting an activation function which is better behaved in the second derivative than tanh. Note how ReLoBRaLo, in combination with early stopping, weakly imposes Pareto optimal updates, as it gradually increases the weights of underperforming terms and eventually leads to a stop of the training process due to a lack of global progress. This is an important property in the context of PINNs because optimising only a subset of terms in the loss can lead to unsatisfactory solutions from a physical perspective.

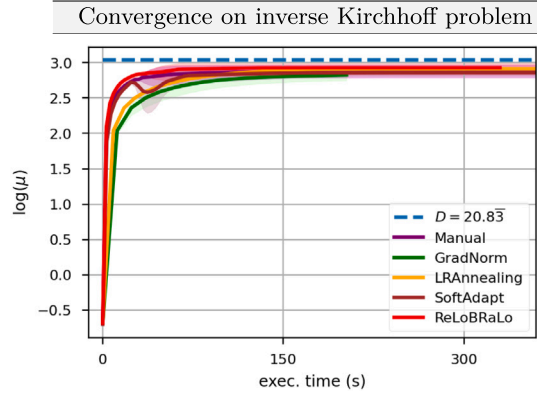
Concerning performance, ReLoBRaLo outperforms the baseline and other algorithms by almost an order of magnitude in accuracy while also yielding a very small standard deviation and hence being very consistent across training runs (cf. Table 4). The results show its effectiveness, even on Kirchhoff's challenging problem with a total of 9 terms (cf. Eq. (21)). Furthermore, the execution times in Table 5 underline the efficiency benefit (up to sixfold speedup) of balancing the loss without gradient statistics, as separate backwards passes for each term become increasingly computationally expensive as the number of terms in the loss function grows. Further conclusions and comparisons are made across different loss balancing methods in Section 9.

For the inverse Kirchhoff problem setting, we select the PDE parameter  $\mu := D$  (i.e. flexural stiffness) to be learned for given data, which we obtained by sampling from the analytically known solution. More specifically, we initialised  $\mu = 0.5$  and tasked the

**Table 4**

Comparison of the median  $L_2$  training and validation loss on Kirchhoff's equation against a baseline of manually chosen scalings. The reported values are the median over four independent runs with identical settings. Additionally, we report the standard deviation over the runs of the best-performing model on the validation loss.

| Kirchhoff                    |             | Baseline            | GradNorm            | LR anneal.          | SoftAdapt           | ReLoBRaLo            |
|------------------------------|-------------|---------------------|---------------------|---------------------|---------------------|----------------------|
| Forward                      | train $f$   | $1.2 \cdot 10^{-7}$ | $5.3 \cdot 10^{-7}$ | $9.1 \cdot 10^{-9}$ | $1.8 \cdot 10^{-8}$ | $6.0 \cdot 10^{-9}$  |
|                              | val $u$     | $1.3 \cdot 10^{-8}$ | $1.7 \cdot 10^{-8}$ | $2.7 \cdot 10^{-9}$ | $2.5 \cdot 10^{-9}$ | $4.0 \cdot 10^{-10}$ |
|                              | std val $u$ | $3.9 \cdot 10^{-8}$ | $2.2 \cdot 10^{-7}$ | $1.0 \cdot 10^{-6}$ | $1.9 \cdot 10^{-9}$ | $7.7 \cdot 10^{-10}$ |
| Inverse<br>$D = 20.8\bar{3}$ | val $\mu$   | 2.1                 | 3.6                 | 6.0                 | 9.5                 | $3.2 \cdot 10^{-2}$  |
|                              | std $\mu$   | 1.6                 | 4.7                 | 0.8                 | 4.9                 | $2.9 \cdot 10^{-2}$  |



**Fig. 7.** Approximation of the true PDE parameter value  $D$  (dashed line) for the inverse problem setting of Kirchhoff plate bending. Reported values are the mean (solid line) and standard deviation (shaded area) of four independent runs.

network with approximating  $D = 20.8\bar{3}$ . Given the large disparity between the initialisation and the target, we empirically found using two separate optimisers beneficial in this case, where one optimiser is used for updating the network's parameters  $\theta$  and a different one for updating the PDE parameter  $\mu$ . Differently from Burgers' equation, ReLoBRaLo also sets a new benchmark in Kirchhoff's inverse problem, both in accuracy as well as convergence speed, cf. Fig. 7 and Table 5.

### 8.3. Helmholtz equation

The Helmholtz equation represents a time-independent form of the wave equation and arises in many physical and engineering problems such as acoustics and electromagnetism [74]. The equation has the form:

$$\Delta u(x, y) + k^2 u(x, y) = f(x, y), \quad x, y \in [-1, 1]^2 \quad (22)$$

where  $k$  is the wave number. This represents a common problem to benchmark PINNs and possesses an analytical solution in combination with Dirichlet boundaries:

$$\begin{aligned} f(x, y) &= (-\pi^2 - (4\pi)^2 + k^2) \sin(\pi x) \sin(4\pi y) \\ u(x, y) &= \sin(\pi x) \sin(4\pi y) \end{aligned} \quad (23)$$

$$u(-1, y) = u(1, y) = u(x, -1) = u(x, 1) = 0$$

Both the  $x_1$  and  $x_2$  input variables are bounded below by  $-1$  and bounded above by  $1$ . Therefore, the boundary conditions add four terms to the loss function of the *forward problem*, resulting in a 5-term total physics-informed loss:

$$\begin{aligned} \mathcal{L}^{(i)} &= \frac{\lambda_0}{|\hat{\Omega}|} \sum_{x, y \in \hat{\Omega}} \left\| \Delta U(x, y; \theta) + k^2 U(x, y; \theta) - f(x, y) \right\|_2^2 \\ &+ \sum_{i=1}^4 \frac{\lambda_i}{|\hat{\Gamma}_i|} \sum_{x, y \in \hat{\Gamma}_i} \|U(x, y; \theta)\|_2^2 \end{aligned} \quad (24)$$

where  $U(x, y; \theta)$  is the parameterisation of the latent function  $\hat{u}(x, y)$  using a neural network with parameters  $\theta$ .

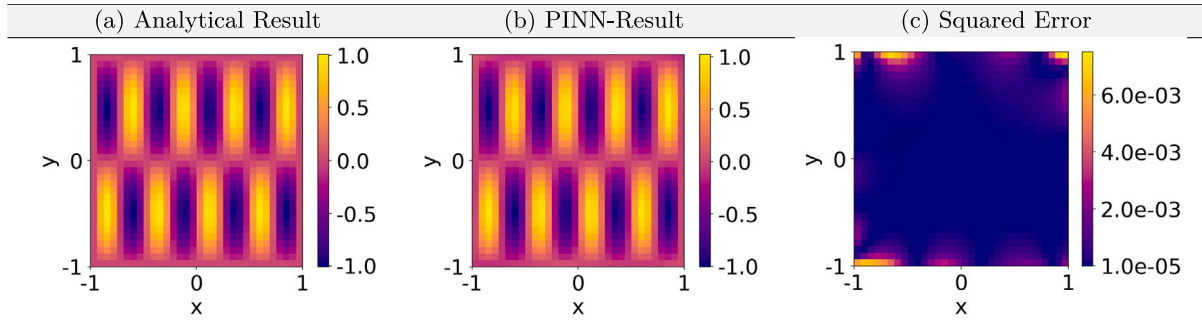
After a successful convergence of the PINNs training, we obtain the results displayed in Fig. 8(b) and compare it to the analytically available solution displayed in Fig. 8(a). A plot of the squared difference in  $u$  as given by the analytical and PINNs results is shown in Fig. 8(c) and delivers a negligible max error. The final algorithm settings are reported in Table 8.

The Helmholtz equation reveals a limitation of our basic loss balancing approach and motivates the introduction of the random lookback. GradNorm and Learning Rate Annealing both achieve impressive results and substantially outperform the baseline as

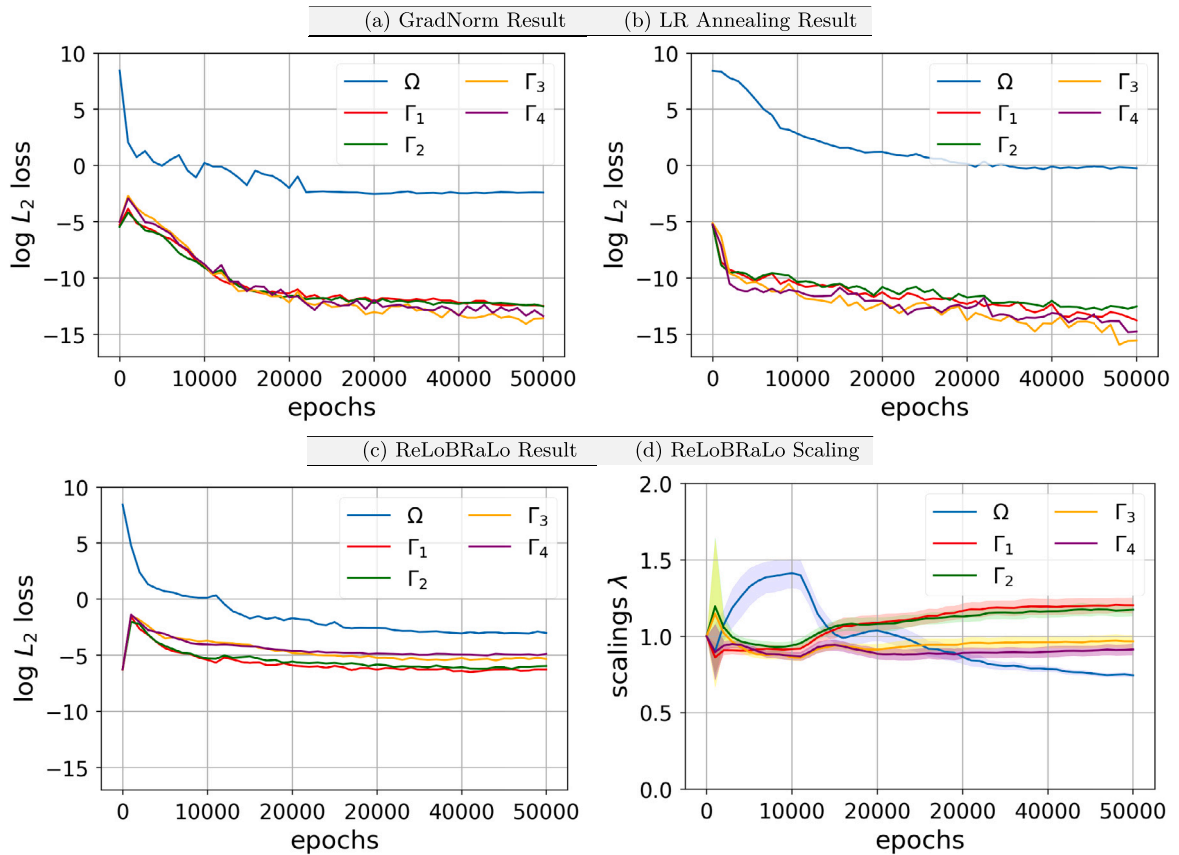
**Table 5**

Median computational overhead  $\Delta T_{co}$  (in s) per 1'000 optimisation steps compared to using no balancing scheme (17.3 s) on Kirchhoff's equation.

|                     | GradNorm | LR ann. | SoftAdapt | ReLoBRaLo |
|---------------------|----------|---------|-----------|-----------|
| $\Delta T_{co}$ [s] | 128.6    | 139.7   | 20.2      | 22.5      |



**Fig. 8.** Helmholtz's problem: (a) analytical reference solution, (b) PINNs results predicted with a fully-connected network consisting of two layers and 128 nodes each, and (c) squared error.



**Fig. 9.** Median of the  $\log L_2$  loss over multiple training runs on the Helmholtz's equation using (a) GradNorm, (b) Learning Rate Annealing and (c) ReLoBRaLo, as well as the mean and variance of scalings calculated by ReLoBRaLo (d).

well as ReLoBRaLo with  $\mathbb{E}[\rho] = 1$  in terms of  $L_2$  accuracy for the BC terms, cf. Fig. 9. This is likely due to the considerable initial difference in magnitudes between the governing equation and the boundary conditions. Furthermore, the high values of  $\alpha$ , necessary for “remembering” the deteriorations longer, induce a latency between the increase of a term's loss until the scaling  $\lambda$  reacts accordingly (cf. Fig. 9(d)). On the other hand, GradNorm and Learning Rate Annealing do not succeed in decreasing the

**Table 6**

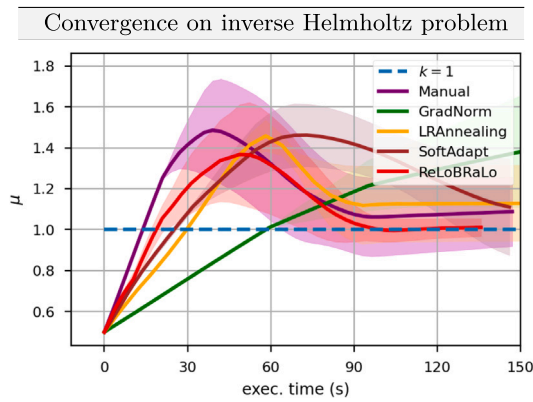
Comparison of the median  $L_2$  training and validation loss on Helmholtz's equation against a baseline of manually chosen scalings. The reported values are the median over four independent runs with identical settings. Additionally, we report the standard deviation over the runs of the best-performing model on the validation loss.

| Helmholtz          |             | Baseline            | GradNorm            | LR anneal.          | SoftAdapt           | ReLoBRaLo           |
|--------------------|-------------|---------------------|---------------------|---------------------|---------------------|---------------------|
| Forward            | train $f$   | $1.4 \cdot 10^{-2}$ | $7.1 \cdot 10^{-2}$ | $2.7 \cdot 10^{-1}$ | $9.5 \cdot 10^{-3}$ | $4.7 \cdot 10^{-3}$ |
|                    | val $u$     | $7.1 \cdot 10^{-2}$ | $5.6 \cdot 10^{-6}$ | $1.4 \cdot 10^{-5}$ | $1.6 \cdot 10^{-3}$ | $2.6 \cdot 10^{-5}$ |
|                    | val std $u$ | $8.1 \cdot 10^{-3}$ | $1.9 \cdot 10^{-5}$ | $7.6 \cdot 10^{-5}$ | $1.5 \cdot 10^{-3}$ | $8.2 \cdot 10^{-5}$ |
| Inverse<br>$k = 1$ | val $\mu$   | $2.7 \cdot 10^{-3}$ | $1.5 \cdot 10^{-1}$ | $5.1 \cdot 10^{-2}$ | $9.1 \cdot 10^{-2}$ | $3.7 \cdot 10^{-4}$ |
|                    | std $\mu$   | $5.0 \cdot 10^{-2}$ | $3.6 \cdot 10^{-1}$ | $7.2 \cdot 10^{-2}$ | $2.1 \cdot 10^{-2}$ | $2.5 \cdot 10^{-4}$ |

**Table 7**

Median computational overhead  $\Delta T_{co}$  (in s) per 1'000 optimisation steps compared to using no balancing scheme (4.8 s) on Helmholtz's equation.

|                     | GradNorm | LR ann. | SoftAdapt | ReLoBRaLo |
|---------------------|----------|---------|-----------|-----------|
| $\Delta T_{co}$ [s] | 10.4     | 6.7     | 5.0       | 5.2       |



**Fig. 10.** Approximation of the true PDE parameter value  $k$  (dashed line) in the inverse problem setting of Helmholtz's equation. Reported values are the mean (solid line) and standard deviation (shaded area) of four independent runs.

$L_2$  error as much as ReLoBRaLo for the governing equation term. Fig. 9 shows that both GradNorm and Learning Rate Annealing focus on improving the boundary conditions right from the beginning of training, whereas ReLoBRaLo with  $\mathbb{E}[\rho] = 1$  counters the initial deterioration, but eventually “forgets” and instead focuses on the more dominant governing equation. This is also reflected in the discrepancy between the training and validation loss: GradNorm and Learning Rate Annealing have a higher training loss than ReLoBRaLo but still exceed at approximating the underlying function (cf. Table 6). This triggered further investigation on the saudade and temperature parameters as described in the remainder of the next section.

For the inverse Helmholtz problem setting, we select the wave number  $k$  to be learned for given data, which we obtained by sampling from the analytically known solution. Furthermore, we initialised  $\mu = 0.5$  and tasked the network with approximating  $k = 1$ . In the Helmholtz inverse problem setting similar to the inverse Burgers problem, just one optimiser was chosen for updating the network's parameters  $\theta$  together with the PDE parameter  $\mu$ . ReLoBRaLo also sets a new benchmark for Helmholtz's inverse problem, both in accuracy as well as convergence speed, cf. Fig. 10 and Table 7.

#### 8.4. Allen–Cahn equation

The Allen–Cahn equation is a nonlinear partial differential equation, significantly used for reaction–diffusion modelling in the field of mathematical physics. The Allen–Cahn equation characterises the phase separation phenomena occurring in e.g. multi-component alloys, including order–disorder transitions.

Within this paper, we consider the Allen–Cahn equation:

$$u_t - 0.0001u_{xx} + 5u^3 - 5u = 0, \quad x \in [-1, 1], \quad t \in [0, 1] \quad (25)$$

**Table 8**  
Final choices of hyperparameters for architecture and training settings.

|                            | Burgers   | Kirchhoff | Helmholtz | Allen-Cahn        |
|----------------------------|-----------|-----------|-----------|-------------------|
| Learn. Rate $l_r$          | $10^{-3}$ | $10^{-3}$ | $10^{-3}$ | $10^{-3}$         |
| Layers $d_K$               | 4         | 4         | 2         | 4                 |
| Width $w_K$                | 256       | 360       | 256       | 256               |
| Exp. Decay $\alpha$        | 0.999     | 0.999     | 0.99      | 0.99              |
| Temp. $\mathcal{T}$        | $10^{-1}$ | $10^{-2}$ | $10^{-5}$ | $5 \cdot 10^{-1}$ |
| Saudade $\mathbb{E}[\rho]$ | 0.9999    | 0.9999    | 0.99      | 0.9               |
| Activation $\sigma$        | $\tanh$   | $\tanh$   | $\tanh$   | $\tanh$           |

**Table 9**  
MSE on the Allen-Cahn equation against the reference solution using different loss balancing schemes.

| PDE         | Baseline | GradNorm | LR anneal. | SoftAdapt | ReLoBRaLo       |
|-------------|----------|----------|------------|-----------|-----------------|
| AllenCahn1D | 1.65e-01 | N/A      | 1.22e-01   | 2.20e-01  | <b>8.55e-02</b> |

**Table 10**  
MSE on the Burgers2D equation against the reference solution using different loss balancing schemes.

| PDE       | Baseline | GradNorm | LR anneal. | SoftAdapt | ReLoBRaLo       |
|-----------|----------|----------|------------|-----------|-----------------|
| Burgers2D | 1.78e-01 | 2.38e-01 | 1.09e-01   | 2.29e-01  | <b>2.29e-01</b> |

along with periodic boundary conditions:

$$\begin{aligned}
 u(0, x) &= x^2 \cos(\pi x) \\
 u(t, -1) &= u(t, 1) \\
 u_x(t, -1) &= u_x(t, 1)
 \end{aligned} \tag{26}$$

For the reference solution, we compare against the results obtained using a polynomial interpolant with Chebyshev points and an exponential time differencing Runge–Kutta scheme [75]. The results are provided in Table 9 and show, that ReLoBRaLo significantly outperforms other loss balancing methods, while LR annealing follows second.

### 8.5. PINNacle benchmark

Finally, we evaluate our loss balancing scheme on a broader range of problems, leveraging the PINNacle benchmark [29], comprising many distinct problems with PDEs from various domains in the forward and inverse setting, including heat conduction, fluid dynamics, biology, and electromagnetics.

#### 2D coupled Burgers equation (Burgers 2D)

The 2D Coupled Burgers equation is given by

$$u_t + u \cdot \nabla u - \nu \Delta u = 0, \tag{27}$$

$$u(0, y, t) = u(L, y, t), \quad u(x, 0, t) = u(x, L, t), \tag{28}$$

$$\{x, y\} \in [0, L], \quad t \in [0, T], \tag{29}$$

The domain is defined as

$$(x, y, t) \in \Omega \equiv [0, L]^2 \times [0, 1]. \tag{30}$$

The initial conditions are given by

$$w(x, y) = \sum_{i=-L}^L \sum_{j=-L}^L \left[ a_{ij} \sin(2\pi(ix + jy)) + b_{ij} \cos(2\pi(ix + jy)) \right], \tag{31}$$

$$u(x, y, 0) = 2w(x, y) + c \tag{32}$$

where  $a, b, c \sim \mathcal{N}(0, 1)$ . The parameters are chosen  $L = 4$ ,  $T = 1$ ,  $\nu = 0.001$ .

The results are provided in Table 10 and show, that ReLoBRaLo is most performant compared to other loss balancing methods together with SoftAdapt.

**Table 11**

MSE on the Poisson2D Classic equation against the reference solution using different loss balancing schemes.

| PDE               | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|-------------------|----------|----------|-----------------|-----------|-----------|
| Poisson2D Classic | 1.14e-01 | 2.05e-01 | <b>3.60e-05</b> | 1.22e-01  | 9.71e-02  |

**Table 12**

MSE on the Poisson–Boltzmann (Helmholtz) 2D Irregular Geometry (Poisson2d-CG) equation against the reference solution using different loss balancing schemes.

| PDE          | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|--------------|----------|----------|-----------------|-----------|-----------|
| Poisson2d-CG | 1.17e-01 | 2.18e-01 | <b>1.62e-02</b> | 1.72e-01  | 1.15e-01  |

### Poisson 2D classic (Poisson2d-C)

The Poisson 2D equation is given by

$$-\Delta u = 0. \quad (33)$$

The domain is a rectangle minus four circles  $\Omega = \Omega_{\text{rec}} \setminus R_i$ , where  $\Omega_{\text{rec}} = [-0.5, 0.5]^2$  is the rectangle and  $R_i$  denotes four circular areas:

$$R_1 = \{(x, y) : (x - 0.3)^2 + (y - 0.3)^2 \leq 0.1^2\}, \quad (34)$$

$$R_2 = \{(x, y) : (x + 0.3)^2 + (y - 0.3)^2 \leq 0.1^2\}, \quad (35)$$

$$R_3 = \{(x, y) : (x - 0.3)^2 + (y + 0.3)^2 \leq 0.1^2\}, \quad (36)$$

$$R_4 = \{(x, y) : (x + 0.3)^2 + (y + 0.3)^2 \leq 0.1^2\}. \quad (37)$$

The boundary condition is

$$u = 0, \quad x \in \partial R_i, \quad (38)$$

$$u = 1, \quad x \in \partial \Omega_{\text{rec}}. \quad (39)$$

The results are provided in Table 11 and show, that ReLoBRaLo outperforms almost all other loss balancing methods yet is dominated by LR annealing.

### Poisson–Boltzmann (Helmholtz) 2D irregular geometry (Poisson2d-CG)

The Poisson–Boltzmann (Helmholtz) 2D equation is given by

$$-\Delta u + k^2 u = f(x, y). \quad (40)$$

The function  $f(x, y)$  is defined as

$$f(x) = A \cdot \left( \sum_i \mu_i^2 + x_i^2 \right) \sin(\mu_1 \pi x_1) \sin(\mu_2 \pi x_2). \quad (41)$$

The domain is  $[-1, 1]^2$ , and the boundary conditions are

$$u = 0.2, \quad x \in \partial \Omega_{\text{rec}}, \quad (42)$$

$$u = 1, \quad x \in \partial \Omega_{\text{circle}}. \quad (43)$$

Parameter references are

$$\mu_1 = 1, \quad \mu_2 = 4, \quad k = 8, \quad A = 10. \quad (44)$$

The domain is  $[-1, 1]^2$  with several circles removed. The circles  $\Omega_{\text{circle}} = \bigcup_{i=1}^4 R_i$  are:

$$R_1 = \{(x, y) : (x - 0.5)^2 + (y - 0.5)^2 \leq 0.2^2\}, \quad (45)$$

$$R_2 = \{(x, y) : (x - 0.4)^2 + (y + 0.4)^2 \leq 0.4^2\}, \quad (46)$$

$$R_3 = \{(x, y) : (x + 0.2)^2 + (y + 0.7)^2 \leq 0.1^2\}, \quad (47)$$

$$R_4 = \{(x, y) : (x + 0.6)^2 + (y - 0.5)^2 \leq 0.3^2\}. \quad (48)$$

Zahlen ausfüllen The results are provided in Table 12 and show, that ReLoBRaLo significantly outperforms other loss balancing methods, while LR annealing follows second.

**Table 13**

MSE on the Poisson 3D equation with two domains (Poisson3d-CG) equation against the reference solution using different loss balancing schemes.

| PDE          | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|--------------|----------|----------|-----------------|-----------|-----------|
| Poisson3d-CG | 3.32e-02 | 4.20e-02 | <b>4.62e-04</b> | 8.19e-03  | 3.23e-02  |

**Table 14**

MSE on the Poisson2D ManyArea equation against the reference solution using different loss balancing schemes.

| PDE                | Baseline | GradNorm        | LR anneal. | SoftAdapt | ReLoBRaLo |
|--------------------|----------|-----------------|------------|-----------|-----------|
| Poisson2D ManyArea | 1.31e+01 | <b>1.21e+01</b> | 1.52e+01   | 1.33e+01  | 1.31e+01  |

### Poisson 3D complex geometry with two domains (Poisson3d-CG)

The Poisson 3D equation with two domains is given by

$$-\mu_i \Delta u + k_i^2 u = f(x, y, z), \quad i = 1, 2. \quad (49)$$

The function  $f(x, y, z)$  is defined as

$$f(x, y, z) = A_1 \cdot \frac{\exp(\sin(m_1 \pi x) \sin(m_2 \pi y) + \sin(m_3 \pi z))}{x^2 + y^2 + z^2 + 1} (x^2 + y^2 + z^2 - 1) + A_2 \sin(m_1 \pi x) \sin(m_2 \pi y) \sin(m_3 \pi z). \quad (50)$$

The coefficients are defined as

$$\mu = \mu_1, \quad k = k_1, \quad x \in \Omega_1, \quad \mu = \mu_2, \quad k = k_2, \quad x \in \Omega_2. \quad (51)$$

The boundary condition is

$$\frac{\partial u}{\partial n} = 0, \quad x \in \partial\Omega. \quad (52)$$

The domains and other parameters are defined as follows:

$$\Omega_1 = [0, 1] \times [0, 1] \times [0, 0.5] \setminus \bigcup_{i=1}^4 R_i, \quad (53)$$

$$\Omega_2 = [0, 1] \times [0, 1] \times [0.5, 1] \setminus \bigcup_{i=1}^4 R_i. \quad (54)$$

The circular regions  $R_i$  are

$$R_1 = \{(x, y) : (x - 0.4)^2 + (y - 0.3)^2 + (z - 0.6)^2 \leq 0.2^2\}, \quad (55)$$

$$R_2 = \{(x, y) : (x - 0.6)^2 + (y - 0.7)^2 + (z - 0.6)^2 \leq 0.2^2\}, \quad (56)$$

$$R_3 = \{(x, y) : (x - 0.2)^2 + (y - 0.8)^2 + (z - 0.7)^2 \leq 0.1^2\}, \quad (57)$$

$$R_4 = \{(x, y) : (x - 0.6)^2 + (y - 0.2)^2 + (z - 0.3)^2 \leq 0.1^2\}. \quad (58)$$

Other parameters are

$$m_1 = 1, \quad m_2 = 10, \quad m_3 = 5, \quad \mu_1 = 1, \quad \mu_2 = 1, \quad k_1 = 8, \quad k_2 = 10, \quad A_1 = 20, \quad A_2 = 100. \quad (59)$$

The results are provided in Table 13 and show, that ReLoBRaLo outperforms almost all other loss balancing methods yet is dominated by LR annealing.

### 2D Poisson equation with many subdomains (Poisson2d-MS)

The PDE and boundary condition is given by

$$-\nabla \cdot (a(x) \nabla u) = f(x, y), \quad x \in \Omega, \quad (60)$$

$$\frac{\partial u}{\partial n} + u = 0, \quad x \in \partial\Omega. \quad (61)$$

Here the domain is  $(x, y) \in \Omega = [-10, 10]^2$ . We divide the whole domain into many small squares, and  $a(x)$  is a piecewise linear function in each square and can be obtained from [29].

The results are provided in Table 14 and show, that ReLoBRaLo slightly outperforms almost all other loss balancing methods yet is dominated by GradNorm.

**Table 15**

MSE on the Heat2D Varying Coefficients equation against the reference solution using different loss balancing schemes.

| PDE                | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|--------------------|----------|----------|-----------------|-----------|-----------|
| Heat2D VaryingCoef | 2.89e-03 | 3.96e-02 | <b>1.85e-03</b> | 5.61e-02  | 3.32e-03  |

**Table 16**

MSE on the Heat2D Multiscale equation against the reference solution using different loss balancing schemes.

| PDE               | Baseline | GradNorm | LR anneal. | SoftAdapt       | ReLoBRaLo |
|-------------------|----------|----------|------------|-----------------|-----------|
| Heat2D Multiscale | 3.10e-06 | 2.38e-05 | 2.25e-04   | <b>1.17e-06</b> | 3.69e-06  |

### 2D heat equation with varying coefficients (Heat2d-VC)

The 2D heat equation with a varying source is given by

$$\frac{\partial u}{\partial t} - \nabla(a(x)\nabla u) = f(x, t). \quad (62)$$

The domain is  $\Omega \times T = [0, 1]^2 \times [0, 5]$ . The function  $a(x)$  is chosen similarly to Darcy flow but with an exponential GRF. The function  $f(x, t)$  is defined as

$$f(x, t) = A \sin(m_1 \pi x) \sin(m_2 \pi y) \sin(m_3 \pi t). \quad (63)$$

With  $A = 200$ ,  $m_1 = 1$ ,  $m_2 = 5$ ,  $m_3 = 1$ . The initial and boundary conditions are

$$u(x, y, 0) = 0, \quad x \in \Omega, \quad (64)$$

$$u(x, y, t) = 0, \quad x \in \partial\Omega. \quad (65)$$

The results are provided in Table 15 and show, that ReLoBRaLo outperforms most loss balancing methods but is dominated by LR annealing. The ground truth data were obtained using COMSOL [29].

### 2D heat multi-scale (Heat2d-MS)

The 2D heat multi-scale equation is given by

$$\frac{\partial u}{\partial t} - \frac{1}{(500\pi)^2} u_{xx} - \frac{1}{\pi^2} u_{yy} = 0, \quad (66)$$

with domain  $\Omega \times T = [0, 1]^2 \times [0, 5]$ .

The initial and boundary conditions are

$$u(x, y, 0) = \sin(20\pi x) \sin(\pi y), \quad x \in \Omega, \quad (67)$$

$$u(x, y, t) = 0, \quad x \in \partial\Omega. \quad (68)$$

The results are provided in Table 16 and show, that ReLoBRaLo outperforms most other loss balancing methods, yet is dominated by SoftAdapt.

### 2D heat complex geometry (heat exchanger, Heat2d-CG)

The 2D heat equation for a complex geometry is given by

$$\frac{\partial u}{\partial t} - \Delta u = 0. \quad (69)$$

The domain is defined as  $\Omega \times T = [-8, 8] \times [-12, 12] \setminus \bigcup_i R_i \times [0, 3]$ .

The boundary condition is

$$-n \cdot (-c \nabla u) = g - qu. \quad (70)$$

with  $c = 1$ . The positions of large circles are

$$(\pm 4, \pm 3), \quad (\pm 4, \pm 9), \quad (0, 0), \quad (0, \pm 6), \quad r = 1, \quad (71)$$

with  $g = 5$  and  $q = 1$ . The positions of small circles are

$$(\pm 3.2, \pm 6), \quad (\pm 3.2, 0), \quad r = 0.4, \quad (72)$$

with  $g = 1$  and  $q = 1$ . For the rectangular boundary conditions,  $g = 0$  and  $q = 1$ .

The results are provided in Table 17 and show, that ReLoBRaLo outperforms most other loss balancing methods, yet is dominated by LR annealing.

**Table 17**

MSE on the Heat2D Complex Geometry equation against the reference solution using different loss balancing schemes.

| PDE                    | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|------------------------|----------|----------|-----------------|-----------|-----------|
| Heat2D ComplexGeometry | 7.87e-05 | 4.45e-04 | <b>2.76e-05</b> | 5.13e-04  | 1.09e-04  |

**Table 18**

MSE on the Heat2D Long Time equation against the reference solution using different loss balancing schemes.

| PDE             | Baseline | GradNorm | LR anneal. | SoftAdapt | ReLoBRaLo       |
|-----------------|----------|----------|------------|-----------|-----------------|
| Heat2D LongTime | 1.14e+00 | 1.14e+00 | 1.14e+00   | 1.28e+00  | <b>1.13e+00</b> |

**Table 19**

MSE on the NS2D Lid-Driven Flow equation against the reference solution using different loss balancing schemes.

| PDE            | Baseline | GradNorm | LR anneal. | SoftAdapt | ReLoBRaLo       |
|----------------|----------|----------|------------|-----------|-----------------|
| NS2D LidDriven | 1.49e-05 | 1.89e-05 | N/A        | 2.23e-05  | <b>1.26e-05</b> |

### 2D heat long time (Heat2d-LT)

The governing PDE is

$$\frac{\partial u}{\partial t} = 0.001 \Delta u + 5 \sin(ku^2) \left( 1 + 2 \sin\left(\frac{\pi t}{4}\right) \right) \sin(m_1 \pi x) \sin(m_2 \pi y). \quad (73)$$

with domain  $\Omega \times T = [0, 1]^2 \times [0, 100]$ ,  $m_1 = 4$ ,  $m_2 = 2$ ,  $k = 1$ .

The initial and boundary conditions are given by

$$u(x, y, 0) = \sin(4\pi x) \sin(3\pi y), \quad x \in \Omega, \quad (74)$$

$$u(x, y, t) = 0, \quad x \in \partial\Omega. \quad (75)$$

The results are provided in Table 18 and show, that ReLoBRaLo significantly outperforms other loss balancing methods.

### 2D NS lid-driven flow (NS2d-C)

The PDE is given by

$$u \cdot \nabla u + \nabla p - \frac{1}{Re} \Delta u = 0, \quad x \in \Omega, \quad (76)$$

$$\nabla \cdot u = 0, \quad x \in \Omega. \quad (77)$$

The domain is  $\Omega = [0, 1]^2$ . The top boundary is  $\Gamma_1$ , and the left, right, and bottom boundary is  $\Gamma_2$ .

The boundary conditions are

$$u(x) = (4x(1-x), 0), \quad x \in \Gamma_1, \quad (78)$$

$$u(x) = (0, 0), \quad x \in \Gamma_2, \quad (79)$$

$$p(x) = 0, \quad x = (0, 0). \quad (80)$$

where in this example, we chose the Reynolds number  $Re = 100$ .

The results are provided in Table 19 and show, that ReLoBRaLo significantly outperforms other loss balancing methods.

### 2D back step flow (NS2d-CG)

The equations and boundary conditions are given by

$$u \cdot \nabla u + \nabla p - \frac{1}{Re} \Delta u = 0, \quad x \in \Omega, \quad (81)$$

$$\nabla \cdot u = 0, \quad x \in \Omega. \quad (82)$$

The domain is defined as  $\Omega = [0, 4] \times [0, 2] \setminus [0, 2] \times [1, 2] \cup R_i$  (excluding the top-left quarter). The inlet velocity is given by  $u_{\text{in}} = 4y(1-y)$ , the outlet pressure is  $p = 0$ , and the boundary condition is no-slip:  $u = 0$ . For this example, we chose the Reynolds number  $Re = 100$ .

The results are provided in Table 20 and show, that ReLoBRaLo significantly outperforms other loss balancing methods.

**Table 20**

MSE on the NS2D Back Step equation against the reference solution using different loss balancing schemes.

| PDE           | Baseline | GradNorm | LR anneal. | SoftAdapt | ReLoBRaLo       |
|---------------|----------|----------|------------|-----------|-----------------|
| NS2D BackStep | 2.77e-04 | 4.16e-04 | 8.09e-03   | 3.83e-04  | <b>2.57e-04</b> |

**Table 21**

MSE on the NS2D Long Time equation against the reference solution using different loss balancing schemes.

| PDE           | Baseline | GradNorm | LR anneal. | SoftAdapt | ReLoBRaLo       |
|---------------|----------|----------|------------|-----------|-----------------|
| NS2D LongTime | 4.90e+02 | N/A      | 5.10e+02   | 4.87e+02  | <b>4.83e+02</b> |

**Table 22**

MSE on the 1D Wave Equation (Wave1d-C) against the reference solution using different loss balancing schemes.

| PDE    | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|--------|----------|----------|-----------------|-----------|-----------|
| Wave1D | 6.73e-03 | 3.71e-02 | <b>6.43e-06</b> | 4.99e-02  | 2.31e-04  |

### 2D NS with time-dependent forcing (NS2d-T)

The PDE of this case is given by

$$\frac{\partial u}{\partial t} + u \cdot \nabla u + \nabla p - \frac{1}{Re} \Delta u = f(x, y, t), \quad (83)$$

$$\nabla \cdot u = 0. \quad (84)$$

The domain is  $\Omega \times T = ([0, 2] \times [0, 1]) \times [0, 5]$ , and the forcing term  $f(x, y, t)$  can be given as

$$f(x, y, t) = (0, -\sin(7\pi x) \sin(5\pi y) \sin(\pi t)). \quad (85)$$

The boundary condition is no-slip:  $u = 0$ , the outlet pressure is  $p = 0$ , and the left inlet initial condition can be given as an oscillatory form:

$$u(0, y, t) = \sin(\pi y)(A_1 \sin(\pi t) + A_2 \sin(3\pi t) + A_3 \sin(5\pi t)), \quad (86)$$

where  $A_1 = 1, A_2 = 1, A_3 = 1$ .

The initial condition in the domain is

$$u(x, y, 0) = 0. \quad (87)$$

The results are provided in [Table 21](#) and show, that ReLoBRaLo significantly outperforms other loss balancing methods.

### Basic 1D wave equation (Wave1d-C)

The governing PDE is

$$u_{tt} - 4u_{xx} = 0. \quad (88)$$

The domain is  $\Omega \times T = [0, 1] \times [0, 1]$ . The boundary conditions are

$$u(0, t) = u(1, t) = 0. \quad (89)$$

The initial condition:

$$u(x, 0) = \sin(\pi x) + \frac{1}{2} \sin(4\pi x), \quad (90)$$

$$u_t(x, 0) = 0. \quad (91)$$

The analytical solution of this problem is

$$u(x, t) = \sin(\pi x) \cos(2\pi t) + \frac{1}{2} \sin(4\pi x) \cos(8\pi t). \quad (92)$$

The results are provided in [Table 22](#) and show, that ReLoBRaLo outperforms other loss balancing methods, yet is dominated by LR annealing.

**Table 23**

MSE on the Wave2D Heterogeneous equation against the reference solution using different loss balancing schemes.

| PDE                  | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|----------------------|----------|----------|-----------------|-----------|-----------|
| Wave2D Heterogeneous | 1.94e-01 | 4.21e-01 | <b>1.12e-01</b> | 2.64e-01  | 1.25e-01  |

**Table 24**

MSE on the Wave2D Long Time equation against the reference solution using different loss balancing schemes.

| PDE             | Baseline | GradNorm        | LR anneal. | SoftAdapt | ReLoBRaLo |
|-----------------|----------|-----------------|------------|-----------|-----------|
| Wave2D LongTime | 7.38e+04 | <b>7.05e+04</b> | 7.74e+04   | 7.53e+04  | 7.74e+04  |

**2D wave equation in heterogeneous medium (Wave2d-CG)**

The governing PDE is given by

$$\left[ \nabla^2 - \frac{1}{c(x)} \frac{\partial^2}{\partial t^2} \right] u(x, t) = 0. \quad (93)$$

The domain is  $\Omega = [-1, 1] \times [-1, 1]$ , and the initial condition is

$$u(x, 0) = \exp\left(-\frac{|x - \mu|^2}{2\sigma^2}\right), \quad x \in \Omega, \quad (94)$$

$$\frac{\partial u}{\partial t}(x, 0) = 0, \quad x \in \Omega. \quad (95)$$

The boundary conditions are

$$\frac{\partial u}{\partial n} = 0, \quad x \in \partial\Omega. \quad (96)$$

The parameters are

$$\mu = (-0.5, 0), \quad \sigma = 0.3, \quad c(x) \text{ are generated by a Gaussian random field.} \quad (97)$$

The results are provided in [Table 23](#) and show, that ReLoBRaLo outperforms other loss balancing methods, yet is dominated by LR annealing.

**2D multi-scale long time wave equation (Wave2d-MS)**

The governing PDE is

$$u_{tt} - (u_{xx} + a^2 u_{yy}) = 0. \quad (98)$$

The domain is defined as  $\Omega = [0, 1]^2 \times [0, 100]$  and the boundary and initial conditions are

$$u(x, y, t) = \begin{cases} 0, & \text{bottom} \\ 0, & \text{left} \\ c_2 \sinh(m_2 \pi x) \sinh(n_2 \pi y) \cos(p_2 \pi t), & \text{right} \\ c_1 \sinh(m_1 \pi x) \sinh(n_1 \pi y) \cos(p_1 \pi t), & \text{top} \end{cases} \quad (99)$$

$$\frac{\partial u}{\partial t}(x, y, 0) = 0. \quad (100)$$

The exact solution to this problem is

$$u(x, y, t) = c_1 \sinh(m_1 \pi x) \sinh(n_1 \pi y) \cos(p_1 \pi t) + c_2 \sinh(m_2 \pi x) \sinh(n_2 \pi y) \cos(p_2 \pi t), \quad (101)$$

where  $a = \sqrt{2}$ ,  $m_1 = 3$ ,  $m_2 = 3$ ,  $n_1 = 2$ ,  $n_2 = 2$ ,  $p_1 = 1$ ,  $p_2 = 1$ ,  $c_1 = 1$ ,  $c_2 = 1$ .

The results are provided in [Table 24](#) and show, that ReLoBRaLo is outperformed by GradNorm.**2D diffusion-reaction Gray-Scott model (GS)**

The governing PDE is

$$u_t = \epsilon_1 \Delta u + b(1 - u) - uv^2, \quad (102)$$

$$v_t = \epsilon_2 \Delta v - dv + uv^2. \quad (103)$$

The domain is  $\Omega \times T = [-1, 1]^2 \times [0, 200]$  and parameters are

$$\epsilon_1 = 10^{-5}, \quad \epsilon_2 = 5 \cdot 10^{-6}, \quad d = 0.1, \quad b = 0.04. \quad (104)$$

**Table 25**

MSE on the Gray–Scott equation against the reference solution using different loss balancing schemes.

| PDE               | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|-------------------|----------|----------|-----------------|-----------|-----------|
| GrayScottEquation | 1.00–01  | N/A      | <b>4.32e–03</b> | 1.26e+04  | 1.56e–01  |

**Table 26**

MSE on the Kuramoto–Sivashinsky equation against the reference solution using different loss balancing schemes.

| PDE                         | Baseline | GradNorm | LR anneal.      | SoftAdapt | ReLoBRaLo |
|-----------------------------|----------|----------|-----------------|-----------|-----------|
| KuramotoSivashinskyEquation | 1.24e+00 | N/A      | <b>1.10e+00</b> | 1.21e+00  | 1.17e+00  |

**Table 27**

MSE on the N-Dimensional Poisson Equation (PNd) against the reference solution using different loss balancing schemes.

| PDE       | Baseline | GradNorm | LR anneal. | SoftAdapt | ReLoBRaLo       |
|-----------|----------|----------|------------|-----------|-----------------|
| PoissonND | 1.14e–06 | N/A      | 2.09e–06   | 1.42e–06  | <b>1.35e–07</b> |

The initial conditions are

$$u(x, y, 0) = 1 - \exp(-80((x - 0.05)^2 + (y + 0.02)^2)), \quad (105)$$

$$v(x, y, 0) = \exp(-80((x - 0.05)^2 + (y - 0.02)^2)). \quad (106)$$

The training results are provided in Table 25 and show, that ReLoBRaLo outperforms other loss balancing methods, yet is dominated by LR annealing.

#### Kuramoto–Sivashinsky equation (KS)

The governing PDE is

$$u_t + \alpha u u_x + \beta u_{xx} + \gamma u_{xxx} = 0. \quad (107)$$

The domain is  $\Omega \times T = [0, 2\pi] \times [0, 1]$ . It is especially interesting to note, that errors may increase rapidly in chaotic problems. The parameters are

$$\alpha = \frac{100}{16}, \quad \beta = \frac{100}{16^2}, \quad \gamma = \frac{100}{16^4}. \quad (108)$$

The initial condition is

$$u(x, 0) = \cos(x)(1 + \sin(x)). \quad (109)$$

The training results are provided in Table 26 and show, that ReLoBRaLo outperforms other loss balancing methods, yet is dominated by LR annealing.

#### N-Dimensional Poisson equation (PNd)

The governing PDE is

$$-\Delta u = \frac{\pi^2}{4} \sum_{i=1}^n \sin\left(\frac{\pi}{2} x_i\right). \quad (110)$$

The domain is defined by  $\Omega = [0, 1]^n$ . The exact solution is

$$u = \sum_{i=1}^n \sin\left(\frac{\pi}{2} x_i\right). \quad (111)$$

For this paper, we choose  $n = 5$  following [29]. The training results are provided in Table 27 and show, that ReLoBRaLo outperforms other loss balancing methods.

#### N-Dimensional heat equation (HNd)

The governing PDE is

$$\frac{\partial u}{\partial t} = k \Delta u + f(x, t), \quad x \in \Omega \times [0, 1] \quad (112)$$

$$n \cdot \nabla u = g(x, t), \quad x \in \partial\Omega \times [0, 1] \quad (113)$$

$$u(x, 0) = g(x, 0), \quad x \in \Omega. \quad (114)$$

**Table 28**

MSE on the HeatND equation against the reference solution using different loss balancing schemes.

| PDE    | Baseline | GradNorm | LR anneal. | SoftAdapt | ReLoBRaLo       |
|--------|----------|----------|------------|-----------|-----------------|
| HeatND | 5.74e-09 | N/A      | 4.52e-03   | 5.78e-09  | <b>2.68e-09</b> |

**Table 29**

Comparison of the MSE against the reference solution on the PINNacle benchmark when varying the loss balancing schemes. The last three columns report the ReLoBRaLo hyperparameters used for achieving the reported performance.

| PDE                         | Baseline | GradNorm        | LR anneal.      | SoftAdapt       | ReLoBRaLo       | Hyperparameters |        |        |
|-----------------------------|----------|-----------------|-----------------|-----------------|-----------------|-----------------|--------|--------|
|                             |          |                 |                 |                 |                 | $\alpha$        | $\tau$ | $\rho$ |
| Burgers2D                   | 1.78e-01 | 2.38e-01        | 1.09e-01        | 2.29e-01        | <b>2.29e-01</b> | 0.99            | 0.1    | 0.95   |
| GrayScottEquation           | 1.00-01  | N/A             | <b>4.32e-03</b> | 1.26e+04        | 1.56e-01        | 0.99            | 0.5    | 0.9    |
| KuramotoSivashinskyEquation | 1.24e+00 | N/A             | <b>1.10e+00</b> | 1.21e+00        | 1.17e+00        | 0.99            | 0.1    | 0.999  |
| Heat2D ComplexGeometry      | 7.87e-05 | 4.45e-04        | <b>2.76e-05</b> | 5.13e-04        | 1.09e-04        | 0.99            | 0.1    | 0.999  |
| Heat2D LongTime             | 1.14e+00 | 1.14e+00        | 1.14e+00        | 1.28e+00        | <b>1.13e+00</b> | 0.99            | 0.1    | 0.999  |
| Heat2D Multiscale           | 3.10e-06 | 2.38e-05        | 2.25e-04        | <b>1.17e-06</b> | 3.69e-06        | 0.99            | 0.1    | 0.999  |
| Heat2D VaryingCoef          | 2.89e-03 | 3.96e-02        | <b>1.85e-03</b> | 5.61e-02        | 3.32e-03        | 0.99            | 0.5    | 0.9    |
| HeatND                      | 5.74e-09 | N/A             | 4.52e-03        | 5.78e-09        | <b>2.68e-09</b> | 0.99            | 0.1    | 0.999  |
| NS2D BackStep               | 2.77e-04 | 4.16e-04        | 8.09e-03        | 3.83e-04        | <b>2.57e-04</b> | 0.99            | 0.1    | 0.99   |
| NS2D LidDriven              | 1.49e-05 | 1.89e-05        | N/A             | 2.23e-05        | <b>1.26e-05</b> | 0.99            | 0.1    | 0.99   |
| NS2D LongTime               | 4.90e+02 | N/A             | 5.10e+02        | 4.87e+02        | <b>4.83e+02</b> | 0.99            | 0.5    | 0.9    |
| Poisson2D Classic           | 1.14e-01 | 2.05e-01        | <b>3.60e-05</b> | 1.22e-01        | 9.71e-02        | 0.99            | 0.5    | 0.9    |
| Poisson2d-CG                | 2.89e-03 | 3.96e-02        | <b>1.85e-03</b> | 5.61e-02        | 3.32e-03        | 0.99            | 0.1    | 0.999  |
| Poisson2D ManyArea          | 1.31e+01 | <b>1.21e+01</b> | 1.52e+01        | 1.33e+01        | 1.31e+01        | 0.99            | 0.1    | 0.95   |
| Poisson3D ComplexGeometry   | 3.32e-02 | 4.20e-02        | <b>4.62e-04</b> | 8.19e-03        | 3.23e-02        | 0.99            | 0.5    | 0.9    |
| PoissonBoltzmann2D          | 1.17e-01 | 2.18e-01        | <b>1.62e-02</b> | 1.72e-01        | 1.15e-01        | 0.999           | 0.01   | 0.999  |
| PoissonND                   | 1.14e-06 | N/A             | 2.09e-06        | 1.42e-06        | <b>1.35e-07</b> | 0.99            | 0.1    | 0.999  |
| Wave1D                      | 6.73e-03 | 3.71e-02        | <b>6.43e-06</b> | 4.99e-02        | 2.31e-04        | 0.99            | 0.1    | 0.999  |
| Wave2D Heterogeneous        | 1.94e-01 | 4.21e-01        | <b>1.12e-01</b> | 2.64e-01        | 1.25e-01        | 0.999           | 0.1    | 0.999  |
| Wave2D LongTime             | 7.38e+04 | <b>7.05e+04</b> | 7.74e+04        | 7.53e+04        | 7.74e+04        | 0.99            | 0.5    | 0.9    |

The geometric domain  $\Omega = \{x : |x|_2 \leq 1\}$  is a unit sphere in  $d$ -dimensional space, where in the context of this paper we choose dimension  $d = 5$  and  $k = \frac{1}{d}$ . The two functions are

$$f(x, t) = -\frac{1}{d} |x|_2^2 \exp\left(\frac{1}{2} |x|_2^2 + t\right), \quad (115)$$

$$g(x, t) = \exp\left(\frac{1}{2} |x|_2^2 + t\right). \quad (116)$$

where  $g(x, t)$  is the exact solution of the equation. The results are provided in [Table 28](#) and show, that ReLoBRaLo significantly outperforms other loss balancing methods.

### Results on the PINNacle benchmark

While the Burgers' equation PDE for 1D (as per [Section 8.1](#) is contained in the PINNacle benchmark [\[29\]](#), the specific Helmholtz equation problem [\[17\]](#), the Kirchhoff plate bending problem [\[2\]](#) as well as the Allen-Cahn PDE [\[76\]](#) are not contained and hence were discussed separately in this paper. The remainder of this section reports on the results of these benchmarks as per [Table 29](#).

The results indicate that employing loss-balancing schemes generally enhances the performance of PINNs. Notably, LR-annealing and ReLoBRaLo are particularly effective. LR-annealing demonstrated superior performance on nine of the nineteen problems, while ReLoBRaLo proved to be the best-performing method on seven problems. SoftAdapt, as well as GradNorm, appear to be the least performing algorithms.

An in-detailed analysis of the individual loss-balancing schemes shows that ReLoBRaLo, as well as LR-annealing, frequently appears as the best or near-best method yielding the lowest mean squared errors (MSE) across a broad spectrum of PDEs. This suggests their effectiveness in dynamically adjusting the learning rate to fine-tune the network training process, particularly beneficial for complex or long-term simulations. Interestingly, for Poisson-type (elliptic) as well as wave-type (hyperbolic) PDEs, LR-annealing has better results than ReLoBRaLo, while for Navier-Stokes (NS) equations, ReLoBRaLo is best performing. For heat transfer PDEs, LR-annealing as well as ReLoBRaLo are close competitors for best performance.

ReLoBRaLo achieves competitive results with significantly lower computational overhead than the gradient-based methods. This is particularly relevant given the challenges observed with LR-annealing, which, while effective, can lead to numerical instabilities such as NaN values if not carefully tuned. Specifically, the NS2D LidDriven problem could not be trained using LR-annealing, echoing issues previously mentioned in the PINNacle report [\[29\]](#).

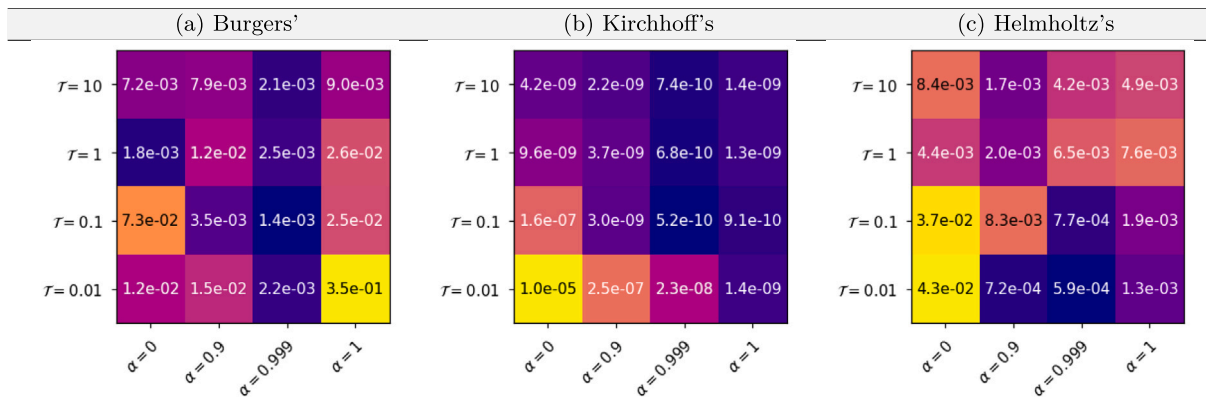


Fig. 11. Ablation of the model's performance when varying  $T$  and  $\alpha$  with  $\mathbb{E}[\rho] = 1$ . The reported values are the median of the log  $L_2$  loss over multiple training runs.

Table 30

Validation loss when varying the expected value of  $\rho$ . The reported values are the median over three independent runs.

| $\mathbb{E}[\rho]$ | Helmholtz           | Burgers             | Kirchhoff            |
|--------------------|---------------------|---------------------|----------------------|
| 0.0                | $2.0 \cdot 10^{-3}$ | $1.0 \cdot 10^{-3}$ | $1.5 \cdot 10^{-09}$ |
| 0.5                | $5.2 \cdot 10^{-5}$ | $9.5 \cdot 10^{-3}$ | $2.7 \cdot 10^{-09}$ |
| 0.9                | $4.0 \cdot 10^{-5}$ | $1.3 \cdot 10^{-3}$ | $2.1 \cdot 10^{-09}$ |
| 0.99               | $2.6 \cdot 10^{-5}$ | $4.9 \cdot 10^{-4}$ | $6.9 \cdot 10^{-10}$ |
| 0.999              | $4.1 \cdot 10^{-5}$ | $3.8 \cdot 10^{-4}$ | $5.6 \cdot 10^{-10}$ |
| 0.9999             | $1.2 \cdot 10^{-4}$ | $1.4 \cdot 10^{-4}$ | $4.0 \cdot 10^{-10}$ |
| 1                  | $8.1 \cdot 10^{-4}$ | $4.7 \cdot 10^{-4}$ | $7.4 \cdot 10^{-10}$ |

## 9. Ablation and sensitivity study

The proposed ReLoBRaLo loss balancing scheme, together with the PINNs architecture, introduces many hyperparameters that have a major influence on performance, efficiency and accuracy. In order to investigate the relations between hyperparameters and find their optimal combinations, we conduct an ablation and sensitivity study w.r.t. the temperature  $T$ , exponential decay rate  $\alpha$ , and expected saudade  $\mathbb{E}[\rho]$  and report its results in this section.

Fig. 11 visualises the models' sensitivity to the exponential decay rate  $\alpha$  and the temperature  $T$ . A larger  $\alpha$  causes the network to "remember" longer, while  $T$  controls how much the scalings "sheer out". Fig. 11(c) shows that Helmholtz's equation benefits most from small values for  $T$ , which turns the balancing more aggressive. This is in line with the findings in the previous section (cf. Section 8.3), where we noted that the large difference in magnitudes between the terms in the loss function caused issues to ReLoBRaLo and that resolute balancing was necessary to avoid the boundary conditions to be neglected. In fact, we found the optimal  $T$  to be  $10^{-5}$  (cf. Table 8). On the other hand, Burgers and Kirchhoff require smoother scalings with a tendency towards higher  $T$  and  $\alpha$ . It is worth noting that all three tasks benefit from the relaxation through the exponential decay, as setting  $\alpha = 1$  always causes a deterioration of the model's performance.

However, the relaxation through the exponential decay induces a new trade-off between making the model remember longer and letting it adapt quickly to changes during training. We, therefore, study the effects of a random lookback through a Bernoulli random variable  $\rho$  (saudade). It allows setting a lower value for  $\alpha$ , thus making the model more flexible, while occasionally "reminding" it of its progress since the start of training  $\mathcal{L}_i^{(0)}$ .

Table 30 summarises the change in performance when varying the expected saudade on all three experiments as a comparison. It is apparent that Helmholtz benefits more from frequent lookbacks, as it hits its best performance at  $\mathbb{E}[\rho] = 0.99$ , whereas Burgers and Kirchhoff only require an expected lookback every 10'000 optimisation steps. Figs. 12 and 3 illustrate the effect of random lookbacks. While the scaling factor's stochasticity increases and makes them less interpretable, it increases the weight of the boundary conditions. The scaled contribution of the boundary conditions consequently leads to a better approximation of the underlying function  $\hat{u}$ .

It is worth noting that the addition of the random lookback improves the accuracy of Helmholtz's equation by more than an order of magnitude while having a lesser, albeit still significant, effect on Burgers' and Kirchhoff's equations.

## 10. Synopsis and outlook

From previous work, we observe that a competitive relationship between physics loss items in the training of PINNs exists and potentially spoils training success, performance or efficiency. This paper investigated different methods aiming at adaptively

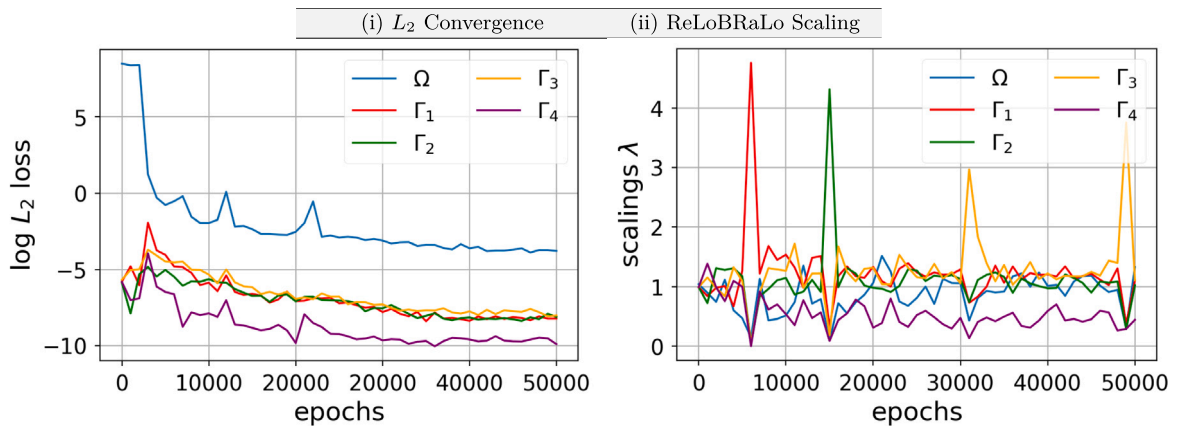


Fig. 12. Example of a single training process of ReLoBRaLo on Helmholtz's equation with  $\alpha = 0.999$ ,  $\mathcal{T} = 10^{-4}$ ,  $\mathbb{E}[\rho] = 0.9999$ .

balancing a loss function consisting of various potentially conflicting objectives as it may arise in scalarised MOO in PINNs. We proposed a novel adaptive loss balancing method by (i) combining the best attributes of existing approaches and (ii) introducing a saudade parameter  $\rho$  to occasionally incorporate historic loss contribution. This forms a new scheme called **Relative Loss Balancing with Random Lookback** (ReLoBRaLo) for selecting bespoke weights in order to combine multiple loss terms for the training of PINNs. The effectiveness and merits of using ReLoBRaLo were then demonstrated empirically by investigating several standard PDEs, including solving the PINNacle benchmark, Helmholtz equation, Burgers' equation, Allen-Cahn equation, and Kirchhoff plate bending equation, and considering both forward problems as well as inverse problems. Our computations showed that ReLoBRaLo is able to consistently outperform the baseline of existing scaling methods (GradNorm, Learning Rate Annealing, SoftAdapt) in terms of accuracy while also being up to six times more computationally efficient (training epochs or wall-clock time). Especially the PINNacle benchmark sample proved ReLoBRaLo to be a strong candidate with a robust performance. Furthermore, the PINNacle benchmark leads to the conclusion that LR annealing and ReLoBRaLo are close competitors, where the nature of the PDE renders either LR annealing or ReLoBRaLo more suitable. The analysis underscores the potential of tailored loss-balancing schemes like LR-annealing and ReLoBRaLo in enhancing the accuracy of PINNs, particularly for simulations involving complex geometries and extensive time spans, despite the challenges of numerical instabilities. Finally, we showed that the adaptively chosen scalings  $\lambda$  can be inspected to learn about the PINNs training process and identify weak points. This allows to make informed decisions in order to improve the framework.

Future research is concerned with the inspection of performance, efficiency, robustness, and scalability of ReLoBRaLo to specific PDE classes such as Poisson and Wave equations, etc. Another interesting aspect of future work is concerned with the investigation of degenerated cases of the loss function, where specifically tailored neural networks by construction satisfy initial or boundary conditions and the loss function hence takes a simpler form. The adoption of Sobolev Training with Sobolev norms or the incorporation of the Mixture-of-Experts approach [41] together with ReLoBRaLo may solve the drawback associated with the high costs involved in estimating the neural network solutions of PDEs.

#### CRedit authorship contribution statement

**Rafael Bischof:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Michael A. Kraus:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

#### Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors did not use any generative AI and AI-assisted technologies.

#### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

#### Data availability

Data will be made available on request.

## References

- [1] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics informed deep learning (part II): Data-driven discovery of nonlinear partial differential equations, 2017, arxiv.org, [arXiv:1711.10566](https://arxiv.org/abs/1711.10566).
- [2] H. Guo, X. Zhuang, T. Rabczuk, A deep collocation method for the bending analysis of Kirchhoff plate, 2021, arXiv e-prints, [arXiv:2102.02617](https://arxiv.org/abs/2102.02617).
- [3] K. Bathe, Finite Element Procedures, in: Finite Element Procedures, No. pt. 2, Prentice Hall, 1996, URL <https://books.google.ch/books?id=AJBRAAAAMAAJ>.
- [4] T. Hughes, The Finite Element Method: Linear Static and Dynamic Finite Element Analysis, in: Dover Civil and Mechanical Engineering, Dover Publications, 2012, URL [https://books.google.ch/books?id=CHH2n\\_qBK0IC](https://books.google.ch/books?id=CHH2n_qBK0IC).
- [5] G. Smith, Numerical Solutions of Partial Differential Equations: Finite Difference Methods, third ed., Oxford University Press, New York, 1985.
- [6] C. Grossmann, H.-G. Roos, M. Stynes, Numerical treatment of partial differential equations, 2007.
- [7] P. Grohs, F. Hornung, A. Jentzen, P. Von Wurstemberger, A proof that artificial neural networks overcome the curse of dimensionality in the numerical approximation of black-scholes partial differential equations, 2018, arXiv preprint [arXiv:1809.02362](https://arxiv.org/abs/1809.02362).
- [8] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, Q. Liao, Why and when can deep-but not shallow-networks avoid the curse of dimensionality: a review, *Int. J. Autom. Comput.* 14 (5) (2017) 503–519.
- [9] E. Haghighat, M. Raissi, A. Moure, H. Gomez, R. Juanes, A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics, *Comput. Methods Appl. Mech. Engrg.* 379 (2021) 113741.
- [10] J.R.R.A. Martins, A. Ning, Engineering Design Optimization, Cambridge University Press, 2021.
- [11] F.-G. Yuan, S.A. Zargar, Q. Chen, S. Wang, Machine learning for structural health monitoring: challenges and opportunities, in: Sensors and Smart Structures Technologies for Civil, Mechanical, and Aerospace Systems 2020, Vol. 11379, International Society for Optics and Photonics, 2020, 1137903.
- [12] Z. Xiang, W. Peng, X. Zheng, X. Zhao, W. Yao, Self-adaptive loss balanced physics-informed neural networks for the incompressible Navier-Stokes equations, 2021, arXiv preprint [arXiv:2104.06217](https://arxiv.org/abs/2104.06217).
- [13] M. Raissi, A. Yazdani, G.E. Karniadakis, Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations, *Science* 367 (6481) (2020) 1026–1030.
- [14] S. Cai, Z. Wang, C. Chrysostomidis, G.E. Karniadakis, Heat transfer prediction with unknown thermal boundary conditions using physics-informed neural networks, in: Fluids Engineering Division Summer Meeting, Vol. 83730, American Society of Mechanical Engineers, 2020, V003T05A054.
- [15] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: A neural tangent kernel perspective, 2020, arXiv e-prints, [arXiv:2007.14527](https://arxiv.org/abs/2007.14527).
- [16] L. McCleenny, U. Braga-Neto, Self-adaptive physics-informed neural networks using a soft attention mechanism, 2020, arXiv e-prints, [arXiv:2009.04544](https://arxiv.org/abs/2009.04544).
- [17] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient pathologies in physics-informed neural networks, 2020, arXiv e-prints, [arXiv:2001.04536](https://arxiv.org/abs/2001.04536).
- [18] C. Zhang, Q. Liao, A. Rakhlin, B. Miranda, N. Golowich, T. Poggio, Theory of Deep Learning IIb: Optimization Properties of SGD, 2018, arXiv e-prints, [arXiv:1801.02254](https://arxiv.org/abs/1801.02254).
- [19] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv e-prints, [arXiv:1412.6980](https://arxiv.org/abs/1412.6980).
- [20] S. Theodoridis, Chapter 5 - online learning: the stochastic gradient descent family of algorithms, in: S. Theodoridis (Ed.), Machine Learning (Second Edition), second ed., Academic Press, 2020, pp. 179–251, URL <https://www.sciencedirect.com/science/article/pii/B9780128188033000143>.
- [21] A. Kendall, Y. Gal, R. Cipolla, Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, 2017, arXiv e-prints, [arXiv:1705.07115](https://arxiv.org/abs/1705.07115).
- [22] Z. Chen, V. Badrinarayanan, C.-Y. Lee, A. Rabinovich, GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks, 2017, arXiv e-prints, [arXiv:1711.02257](https://arxiv.org/abs/1711.02257).
- [23] A.A. Heydari, C.A. Thompson, A. Mehmood, SoftAdapt: Techniques for adaptive loss weighting of neural networks with multi-part loss functions, 2019, arXiv e-prints, [arXiv:1912.12355](https://arxiv.org/abs/1912.12355).
- [24] I.E. Lagaris, A. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Trans. Neural Netw.* 9 (5) (1998) 987–1000.
- [25] I. Lagaris, A. Likas, D. Papageorgiou, Neural-network methods for boundary value problems with irregular boundaries, *IEEE Trans. Neural Netw.* 11 (5) (2000) 1041–1049.
- [26] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, *J. Comput. Phys.* 375 (2018) 1339–1364, [arXiv:1708.07469](https://arxiv.org/abs/1708.07469). URL <https://www.sciencedirect.com/science/article/pii/S0021999118305527>.
- [27] M. Raissi, Deep hidden physics models: Deep learning of nonlinear partial differential equations, *J. Mach. Learn. Res.* 19 (2018) 1–24, [arXiv:1801.06637](https://arxiv.org/abs/1801.06637). URL <https://www.sciencedirect.com/science/article/pii/S0021999117309014>.
- [28] Y. Shin, On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs, *Commun. Comput. Phys.* 28 (5) (2020) 2042–2074, [arXiv:2004.01806](https://arxiv.org/abs/2004.01806).
- [29] Z. Hao, J. Yao, C. Su, H. Su, Z. Wang, F. Lu, Z. Xia, Y. Zhang, S. Liu, L. Lu, J. Zhu, PINNacle: A comprehensive benchmark of physics-informed neural networks for solving PDEs, 2023, <https://dx.doi.org/10.48550/arXiv.2306.08827>, [arXiv:2306.08827](https://arxiv.org/abs/2306.08827).
- [30] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [31] A.D. Jagtap, K. Kawaguchi, G. Em Karniadakis, Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks, *Proc. R. Soc. Lond. Ser. A* 476 (2239) (2020) 20200334, [arXiv:1909.12228](https://arxiv.org/abs/1909.12228).
- [32] A.D. Jagtap, K. Kawaguchi, G.E. Karniadakis, Adaptive activation functions accelerate convergence in deep and physics-informed neural networks, *J. Comput. Phys.* 404 (2020) 109136, [arXiv:1906.01170](https://arxiv.org/abs/1906.01170).
- [33] Y. Kim, Y. Choi, D. Widemann, T. Zohdi, A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder, 2020, arXiv e-prints, [arXiv:2009.11990](https://arxiv.org/abs/2009.11990).
- [34] W. Peng, W. Zhou, J. Zhang, W. Yao, Accelerating physics-informed neural network training with prior dictionaries, 2020, arXiv e-prints, [arXiv:2004.08151](https://arxiv.org/abs/2004.08151).
- [35] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: A deep learning library for solving differential equations, *SIAM Rev.* 63 (1) (2021) 208–228.
- [36] A. Jagtap, E. Kharazmi, G. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems, *Comput. Methods Appl. Mech. Engrg.* 365 (2020) 113028.
- [37] A. Jagtap, G. Karniadakis, Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations, *Commun. Comput. Phys.* 28 (2020) 2002–2041.
- [38] K. Shukla, A.D. Jagtap, G.E. Karniadakis, Parallel Physics-Informed Neural Networks via Domain Decomposition, 2021, arXiv e-prints, [arXiv:2104.10013](https://arxiv.org/abs/2104.10013).
- [39] X. Meng, Z. Li, D. Zhang, G.E. Karniadakis, PPINN: Parareal physics-informed neural network for time-dependent PDEs, *Comput. Methods Appl. Mech. Engrg.* 370 (2020) 113250, URL <https://www.sciencedirect.com/science/article/pii/S0045782520304357>.
- [40] E. Kharazmi, Z. Zhang, G.E. Karniadakis, Hp-VPINNs: Variational physics-informed neural networks with domain decomposition, *Comput. Methods Appl. Mech. Engrg.* 374 (2021) 113547, URL <https://www.sciencedirect.com/science/article/pii/S0045782520307325>.
- [41] R. Bischof, M.A. Kraus, Mixture-of-experts-ensemble meta-learning for physics-informed neural networks, in: Proceedings of 33. Forum Bauinformatik, 2022.

- [42] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Y.W. Teh, M. Titterton (Eds.), Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, in: Proceedings of Machine Learning Research, vol. 9, PMLR, Chia Laguna Resort, Sardinia, Italy, 2010, pp. 249–256, URL <https://proceedings.mlr.press/v9/glorot10a.html>.
- [43] X. Liu, X. Zhang, W. Peng, W. Zhou, W. Yao, A novel meta-learning initialization method for physics-informed neural networks, 2021, arXiv preprint [arXiv:2107.10991](https://arxiv.org/abs/2107.10991).
- [44] S. Chakraborty, Transfer learning based multi-fidelity physics informed deep neural network, J. Comput. Phys. 426 (2021) 109942, URL <https://www.sciencedirect.com/science/article/pii/S0021999120307166>.
- [45] S. Goswami, C. Anitescu, S. Chakraborty, T. Rabczuk, Transfer learning enhanced physics informed neural network for phase-field modeling of fracture, Theor. Appl. Fract. Mech. 106 (2020) 102447, URL <https://www.sciencedirect.com/science/article/pii/S016784421930357X>.
- [46] A. Rajeswaran, C. Finn, S. Kakade, S. Levine, Meta-learning with implicit gradients, 2019, arXiv e-prints, [arXiv:1909.04630](https://arxiv.org/abs/1909.04630).
- [47] K.A. Smith-Miles, Cross-disciplinary perspectives on meta-learning for algorithm selection, ACM Comput. Surv. 41 (1) (2009).
- [48] C. Finn, S. Levine, Meta-learning and universality: Deep representations and gradient descent can approximate any learning algorithm, 2017, arXiv e-prints, [arXiv:1710.11622](https://arxiv.org/abs/1710.11622).
- [49] C. Finn, K. Xu, S. Levine, Probabilistic model-agnostic meta-learning, 2018, arXiv e-prints, [arXiv:1806.02817](https://arxiv.org/abs/1806.02817).
- [50] C. Finn, P. Abbeel, S. Levine, Model-agnostic meta-learning for fast adaptation of deep networks, in: Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML '17, JMLR.org, 2017, pp. 1126–1135.
- [51] A. Nichol, J. Achiam, J. Schulman, On first-order meta-learning algorithms, 2018, arXiv e-prints, [arXiv:1803.02999](https://arxiv.org/abs/1803.02999).
- [52] C. Wu, M. Zhu, Q. Tan, Y. Kartha, L. Lu, A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks, Comput. Methods Appl. Mech. Engrg. 403 (2023) 115671.
- [53] Z. Gao, L. Yan, T. Zhou, Failure-informed adaptive sampling for PINNs, SIAM J. Sci. Comput. 45 (4) (2023) A1971–A1994.
- [54] W.M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, R. Pascanu, Sobolev training for neural networks, in: Advances in Neural Information Processing Systems, Vol. 2017-Decem, 2017, pp. 4279–4288, [arXiv:1706.04859](https://arxiv.org/abs/1706.04859). URL <http://papers.nips.cc/paper/7015-sobolev-training-for-neural-networks>.
- [55] H. Son, J.W. Jang, W.J. Han, H.J. Hwang, Sobolev training for the neural network solutions of PDEs, 2021, arXiv preprint [arXiv:2101.08932](https://arxiv.org/abs/2101.08932).
- [56] N.N. Vlassis, W. Sun, Sobolev training of thermodynamic-informed neural networks for interpretable elasto-plasticity models with level set hardening, Comput. Methods Appl. Mech. Engrg. 377 (2021) 113695.
- [57] N.N. Vlassis, R. Ma, W. Sun, Geometric deep learning for computational mechanics part i: Anisotropic hyperelasticity, Comput. Methods Appl. Mech. Engrg. 371 (2020) 113299.
- [58] M.A. Kraus, Machine Learning Techniques for the Material Parameter Identification of Laminated Glass in the Intact and Post-Fracture State (Ph.D. thesis), Universität der Bundeswehr München, 2019.
- [59] M.A. Kraus, M. Drass, Artificial intelligence for structural glass engineering applications—overview, case studies and future potentials, Glas. Struct. Eng. 5 (3) (2020) 247–285.
- [60] C.L. Wight, J. Zhao, Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks, 2020, arXiv e-prints, [arXiv:2007.04542](https://arxiv.org/abs/2007.04542).
- [61] R. Caruana, Multitask learning, Mach. Learn. 28 (1997).
- [62] D.F. Jones, S.K. Mirrazavi, M. Tamiz, Multi-objective meta-heuristics: An overview of the current state-of-the-art, European J. Oper. Res. 137 (1) (2002) 1–9.
- [63] K.-H. Chang, Chapter 17 - design optimization, in: K.-H. Chang (Ed.), E-Design, Academic Press, Boston, 2015, pp. 907–1000, URL <https://www.sciencedirect.com/science/article/pii/B978012382038900017X>.
- [64] O. Sener, V. Koltun, Multi-task learning as multi-objective optimization, in: S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 31, Curran Associates, Inc., 2018, URL <https://proceedings.neurips.cc/paper/2018/file/432aca3a1e345e339f35a30c8f65edce-Paper.pdf>.
- [65] M. Ruchte, J. Grabocka, Efficient multi-objective optimization for deep learning, 2021, arXiv [abs/2103.13392](https://arxiv.org/abs/2103.13392).
- [66] K. Hornik, M. Stinchcombe, H. White, Universal approximation of an unknown mapping and its derivatives using multilayer feedforward networks, Neural Netw. 3 (5) (1990) 551–560, URL <https://www.sciencedirect.com/science/article/pii/0893608090900056>.
- [67] P. Liashchynskiy, P. Liashchynskiy, Grid search, random search, genetic algorithm: A big comparison for NAS, 2019, arXiv e-prints, [arXiv:1912.06059](https://arxiv.org/abs/1912.06059).
- [68] J. Snoek, H. Larochelle, R.P. Adams, Practical Bayesian optimization of machine learning algorithms, in: Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 2, NIPS '12, Curran Associates Inc., Red Hook, NY, USA, 2012, pp. 2951–2959.
- [69] F. Nogueira, Bayesian optimization: Open source constrained global optimization tool for Python, 2014, URL <https://github.com/fmfn/BayesianOptimization>.
- [70] J. Mockus, V. Tiesis, A. Zilinskas, The Application of Bayesian Methods for Seeking the Extremum, Vol. 2, 2014, pp. 117–129.
- [71] P. Orlandi, The Burgers equation, in: P. Orlandi (Ed.), Fluid Flow Phenomena: A Numerical Toolkit, Springer Netherlands, Dordrecht, 2000, pp. 40–50.
- [72] M.P. Bonkile, A. Awasthi, C. Lakshmi, V. Mukundan, V.S. Aswin, A systematic literature review of Burgers' equation with recent advances, Pramana 90 (6) (2018) 69.
- [73] European Committee for Standardization, EN 1992-1-1 Eurocode 2: Design of Concrete Structures - Part 1-1: General Rules and Rules for Buildings, CEN, Brussels, 2005.
- [74] A. Sommerfeld, Partial Differential Equations in Physics, Academic Press, 1949.
- [75] R. Matthey, S. Ghosh, A novel sequential method to train physics informed neural networks for Allen Cahn and Cahn Hilliard equations, Comput. Methods Appl. Mech. Engrg. 390 (2022) 114474.
- [76] C.L. Wight, J. Zhao, Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks, 2020, [http://dx.doi.org/10.48550/arXiv.2007.04542](https://dx.doi.org/10.48550/arXiv.2007.04542), arXiv e-prints, [arXiv:2007.04542](https://arxiv.org/abs/2007.04542).